



UNIVERSITY OF DERBY
MEDITERRANEAN COLLEGE, THESSALONIKI
School of Computing and Engineering

MSc in Big Data Analytics

**LLM-Based Classification of Move
Smart-Contract Issues in the Sui Ecosystem:
Dataset Consolidation and
Baseline Model Comparison**

Vasileios Bakalis
Student ID: 100827245

Supervisor: Dr. Ioannis Tsioulis

May 2026

Contents

1	Introduction	2
1.1	Project Rationale	3
1.2	Project Aim and Objectives	4
2	Literature Review	4
2.1	Introduction	4
2.2	Foundations of Neural Language Models	4
2.3	Blockchains and Smart Contracts	7
2.4	The Sui Blockchain	8
2.5	The Move Programming Language	9
2.6	Related Work	11
2.7	Conclusions	12
3	Research Methodology	12
3.1	Introduction	12
3.2	Research Strategy	12
3.3	Data Generation Methods	12
3.4	Data Analysis (Preprocessing)	13
3.5	Sampling and Exploratory Analysis	13
3.6	Annotation Protocol and Inter-Rater Considerations	15
3.7	Ethics	16
3.8	Limitations (Threats to Validity)	16
3.9	Conclusions	17
4	Experimental Setup and Findings: Design and Implementation	17
4.1	Introduction	17
4.2	Simple Baselines: TF-IDF + Classical Classifiers	17
4.3	Prompting Baselines	18
4.4	Transformer Fine-Tuning	18
4.5	4-bit LoRA on a 7B Decoder (MLX)	19
4.6	Evaluation Metrics	19
4.7	Design alternatives considered	20
4.8	Implementation and Reproducibility	21
4.9	Conclusions	22
5	Analysis: Testing and Evaluation	22

5.1	Introduction	22
5.2	Test Setup	22
5.3	Simple Baselines	22
5.4	Prompting Baselines	23
5.5	Transformer Fine-Tuning (CodeBERT)	24
5.6	Overall Comparison	24
5.7	External LLM Baseline: Claude Sonnet 4.6	26
5.8	Error Analysis	26
5.9	Conclusions	31
6	Discussion	31
6.1	Introduction	31
6.2	Conclusion	33
7	Conclusions and Recommendations	33
7.1	Conclusions	33
7.2	Recommendations	33
A	Dataset Details	35
B	Per-Class Results	35
C	Hyperparameter Settings	36
D	Progress Log	36
E	Sample Dataset Entries	38
F	Confusion Matrices per Model	41
G	Reproducibility Checklist	42
H	Deployed System	43
I	Glossary	45
J	Compute and Cost Breakdown	46
K	Lessons Learned	47
L	Selected Implementation Listings	49

LLM-Based Classification of Move Smart-Contract Issues in the Sui Ecosystem: Dataset Consolidation and Baseline Model Comparison

Vasileios Bakalis

Mediterranean College / University of Derby
vasilis.mbakalis@gmail.com

Reviewed on OpenReview: <https://github.com/Vbakalis/move-issue-classifier>

Abstract

Move smart contracts on the Sui blockchain can fail in many ways: syntax errors, broken semantics, security holes, or simply stylistic problems that make the code hard to audit. Reliably sorting these issues into categories is useful both for developers fixing their own code and for reviewers triaging incoming pull requests, but Move tooling for this is still thin. This thesis builds a labelled dataset of about 3,300 Move snippets across five categories (*Perfect*, *SecurityError*, *SemanticError*, *StyleError*, *SyntaxError*) and uses it to compare five baseline approaches on a single fixed test split. The approaches sit on a spectrum from cheap to heavy: TF-IDF with a linear classifier, zero-shot and few-shot prompting of a 7B code LLM, full fine-tuning of CodeBERT, LoRA fine-tuning of CodeBERT, and 4-bit LoRA fine-tuning of the same 7B decoder via Apple MLX.

The headline numbers are clear. Prompting Qwen2.5-Coder-7B-Instruct without any training reaches 49.9% accuracy and 36.9% macro-F1 in the zero-shot configuration, climbing to 59.1% accuracy and 51.2% macro-F1 with one example per class. A larger frontier model (Claude Sonnet 4.6, Anthropic) reaches 64.4% accuracy and 64.8% macro-F1 zero-shot but still falls far short of fine-tuning. All prompting configurations sit well below even the lexical TF-IDF baseline, which already achieves 97.2% accuracy and 96.5% macro-F1 with LinearSVC. Fine-tuning closes the remaining gap: full CodeBERT reaches 98.8% accuracy and 99.2% macro-F1, and a LoRA adapter on the same encoder matches or slightly beats it at 99.0% accuracy and 99.3% macro-F1 while updating less than 1% of the weights. The 4-bit MLX LoRA fine-tune of the 7B decoder lands at 97.2% accuracy and 97.8% macro-F1, below the much smaller fine-tuned encoder.

On this dataset, a small task-aligned encoder dominates a quantised 7B decoder, and parameter-efficient adapters give essentially the same quality as full fine-tuning at a fraction of the training cost. Every experiment runs on a single Apple Silicon laptop, and the CodeBERT runs each finish in under ten minutes.

1 Introduction

Sui is a high-performance Layer-1 blockchain whose smart contracts are written in Move (Mysten Labs, 2024). The ecosystem is growing fast, but the tooling that helps developers catch issues in their Move code has not kept up: most Move-specific analysers are rule-based and narrow in scope, and the kind of pre-trained code-understanding models that have become standard for Python or Java are not directly applicable.

Large language models trained on code have proven useful across a wide range of software-engineering tasks, defect detection, code classification, vulnerability spotting, but their behaviour on Move, a comparatively low-resource language with unusual resource and ownership semantics, has not really been studied. This thesis is an attempt to fill that gap with a concrete, reproducible benchmark.

Scope and framing. This work is best read as a reproducible benchmark and systems study for Move-on-Sui issue classification, rather than as an algorithmic innovation. We do not propose a new model architecture, a new training objective, or a new fine-tuning recipe. What we contribute instead is a curated dataset, a clear side-by-side of four method families that span the relevant cost/quality space, and an end-to-end pipeline that runs on a single laptop. The intended audience is the next person who wants to train a Move classifier or who needs a defensible starting point for a Sui-side tooling effort: they should be able to take this thesis, the public artefact bundle on Zenodo, and the accompanying code, and reproduce every number in this document inside a single afternoon.

The contributions are:

1. A consolidated, deduplicated dataset of about 3,300 Move snippets labelled across five issue categories, with stratified train/validation/test splits.
2. A side-by-side comparison of five baselines on that dataset: TF-IDF with classical classifiers (Logistic Regression, LinearSVC, Random Forest); zero-shot and few-shot prompting of Qwen2.5-Coder-7B-Instruct without weight updates; full fine-tuning of CodeBERT with a classification head; LoRA fine-tuning of the same encoder (about 0.94% of the parameters); and 4-bit LoRA fine-tuning of Qwen2.5-Coder-7B-Instruct via Apple MLX, the Apple-Silicon equivalent of QLoRA.
3. A per-class error analysis showing that fine-tuned CodeBERT cleans up the *SemanticError/SyntaxError* confusion that the lexical baseline still makes, and that LoRA matches full fine-tuning at a fraction of the cost. The 125M-parameter encoder also outperforms the 4-bit 7B decoder, which suggests that at this dataset scale architecture fit beats raw parameter count.
4. An end-to-end pipeline that runs on a single Apple Silicon laptop without CUDA, with each CodeBERT run finishing in under ten minutes.

1.1 Project Rationale

It is worth being explicit about the setting we have in mind for this classifier, because everything from the choice of metric to the way we treat the residual error rate depends on it. The classifier is not meant to be the last line of defence in a Move audit. The setting we have in mind is closer to a fast triage filter that sits in front of a human reviewer.

Where the classifier sits. There are three plausible places for a model like this in a Move-on-Sui workflow. The first is the developer’s IDE, where the classifier runs continuously on the buffer the developer is editing and surfaces a small banner whenever the predicted class is not *Perfect*. Latency dominates here: anything that takes longer than a second per snippet is unusable, which is already a strong argument against running a 7B-parameter backbone in this slot. The second is a CI pipeline, where the classifier runs on every push or pull request, batches together the snippets in the diff, and posts a comment that flags any class other than *Perfect*. Throughput matters more than latency here, and a tiny model that processes thousands of snippets in under a minute is the right shape. The third is a dedicated audit pipeline, where a human auditor uses the classifier as a triage layer over a large legacy code base before deciding which modules to read carefully. Throughput matters again, but recall on *SecurityError* matters more than accuracy on *StyleError*, because a missed security issue is far more expensive than a false alarm on whitespace.

What happens after the prediction. A prediction by itself is not a finished product. In every one of the three settings above, the prediction feeds into a human-in-the-loop step. The IDE banner asks the developer whether they want to look at it again before saving. The CI comment becomes a discussion thread on the pull request. The auditor uses the predictions to choose a reading order, not to skip files entirely. The classifier exists to redirect human attention, not to replace it, and that framing matters because it shifts what counts as a good outcome. A model that catches 95% of the real *SecurityError* cases at the cost of a handful of false positives on *StyleError* is, in this setting, a strictly better tool than a model with one extra point of macro-F1 that gets a single *SecurityError* wrong. We do not actually optimise the classifier for that

asymmetric loss in this thesis (we report uniform-cost macro-F1 and accuracy throughout), but the setting it is meant to slot into is one in which the cost of errors is asymmetric.

Why this framing informs the metric and method choices. Three downstream choices follow from this view of the problem. First, we report macro-F1 alongside accuracy because the expensive class (*SecurityError*) is the smallest, and a metric that averages performance across classes uniformly matches the auditor’s view of the world better than one that is dominated by the majority *Perfect* class. Second, we care about cheap inference: a CodeBERT-based fine-tune that runs in tens of milliseconds per snippet on a laptop is a much better fit for the IDE and CI use cases than a 7B-parameter decoder that needs at least a few hundred milliseconds even at 4-bit precision, even when the larger model is slightly more accurate. Third, we evaluate on a held-out test set drawn from the same annotation pipeline, which is appropriate for a tool designed to assist the same kind of reviewer who produced the labels in the first place; a tool meant for fully autonomous deployment would need a separate independently-labelled evaluation set, which we do not have.

1.2 Project Aim and Objectives

The aim, objectives, and intended usage of this project are stated in the introduction above and elaborated in §1.1.

2 Literature Review

2.1 Introduction

This chapter reviews the foundations and prior work that frame the project.

This section gives just enough background to read the rest of the paper: the deep-learning machinery the classifiers are built on (Section 2.2), how blockchains and smart contracts work in general (Section 2.3), what makes Sui different (Section 2.4), and the Move language we are actually classifying code in (Section 2.5). Readers already familiar with all four can skip ahead to Section 2.6.

2.2 Foundations of Neural Language Models

Every method in this thesis except the TF-IDF baseline is built on the same lineage of ideas: pre-trained transformer language models, adapted to a downstream task either through full fine-tuning, parameter-efficient fine-tuning, or in-context prompting. This section gathers the pieces a reader needs in order to follow the methods and results, without reproducing a full deep-learning textbook (Goodfellow et al., 2016).

Neural networks and gradient-based learning. A modern neural network is a parameterised function $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ composed of many elementary differentiable operations: affine maps, element-wise non-linearities, normalisations, and so on. Given a labelled training set $\{(x_i, y_i)\}_{i=1}^N$ and a loss function $\mathcal{L}$ that measures disagreement between predictions and targets, training reduces to solving

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i).$$

The minimisation is carried out by stochastic gradient descent, or in practice by an adaptive variant such as Adam (Kingma & Ba, 2015), in which the gradient $\nabla_{\theta} \mathcal{L}$ is estimated on small mini-batches and the parameters are nudged in the direction that locally reduces the loss. For classification tasks the loss is typically the cross-entropy between the predicted class distribution and a one-hot target; for autoregressive language modelling it is the negative log-likelihood of the next token. Gradients are computed analytically by reverse-mode automatic differentiation, sometimes still called backpropagation. None of this is conceptually new; it dates back to Rumelhart, Hinton and Williams in 1986 and earlier, but two practical changes turned

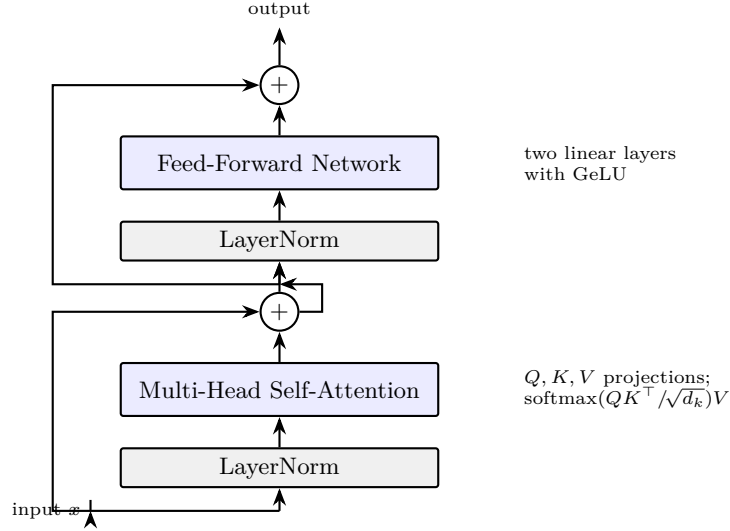


Figure 1: A single Transformer encoder block. Self-attention mixes information across positions, the feed-forward network mixes information across the feature dimension, and residual connections plus LayerNorm keep gradients well-behaved when the block is stacked many times. CodeBERT-base uses 12 such blocks; Qwen2.5-Coder-7B uses 28.

it into the dominant paradigm of the last decade: massive labelled and unlabelled corpora, and the general-purpose, hardware-accelerated tensor compilers that make training a model with hundreds of millions of parameters routine rather than exceptional (Hinton et al., 2006; Bengio & LeCun, 2007).

The Transformer architecture. The transformer (Vaswani et al., 2017) is the architecture underlying essentially every model used in this thesis. Its core idea is to replace recurrence with *self-attention*: given a sequence of token embeddings $X \in \mathbb{R}^{n \times d}$, three learned linear projections produce queries, keys, and values $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and the attention output is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Conceptually, each token’s output is a weighted average of every other token’s value vector, where the weights are similarities between query and key. *Multi-head* attention runs this operation in parallel with multiple independent (W_Q, W_K, W_V) triples and concatenates the results, which lets the model attend to several relations at once: one head might track syntactic dependency, another lexical co-occurrence, and so on. A transformer block stacks a multi-head self-attention layer, a residual connection and layer normalisation, a position-wise feed-forward network (typically two linear maps with a non-linear activation in between), and a second residual + normalisation. The full encoder or decoder is just a stack of such blocks.

Two variants matter for this thesis. *Encoder-only* models such as BERT (Devlin et al., 2019) and its descendants RoBERTa (Liu et al., 2019) and CodeBERT (Feng et al., 2020) run the full sequence through bidirectional attention and produce a fixed-size representation per token; a classification head over the special [CLS] token (or a pooled sequence representation) is the canonical way to use them for tasks like ours. *Decoder-only* models such as the GPT family and the open-weight CodeLlama, StarCoder and Qwen-Coder families use causal (left-to-right) self-attention and are trained to predict the next token; they are naturally suited to generation and few-shot prompting, and we use a 7B-parameter Qwen-Coder checkpoint in exactly that way for the prompting baseline (Section 4.3).

Pre-training, fine-tuning, and transfer learning. The single most important practical observation of the past five years of NLP is that a model trained once on a very large unlabelled corpus learns a general-purpose representation that transfers to many downstream tasks with only a small amount of task-specific

data. For encoder-only models the dominant pre-training objective is *masked language modelling*: a random 15% of the input tokens are replaced by a special [MASK] symbol, and the network is trained to recover them; for decoder-only models it is *causal language modelling*, predicting each token from the prefix that precedes it. After pre-training the model is repurposed in one of three ways. First, *full fine-tuning* updates all parameters on the target task, which is what the CodeBERT-full configuration in Section 4.4 does. Second, *parameter-efficient fine-tuning* freezes the base weights and trains a small number of adapter parameters, which is what the CodeBERT+LoRA and Qwen-7B+LoRA configurations do. Third, *in-context learning* keeps the weights frozen and steers behaviour through prompt engineering alone, which is what the zero-shot and few-shot prompting baselines in Section 4.3 do. All three are post-hoc adaptations of the same pre-trained body of knowledge; they differ only in how much, and which part, of that body is re-shaped to fit the target task.

Tokenisation. Transformers operate on integer sequences, but source code is text. The bridge between the two is a *tokenizer* that splits each input string into a sequence of subword units drawn from a fixed vocabulary. Modern code models almost universally use byte-pair encoding (BPE) (Sennrich et al., 2016) or one of its byte-level variants. BPE starts from individual characters and greedily merges the most frequent adjacent pair into a new token, repeating until the vocabulary reaches its target size. The result is a vocabulary that represents common substrings (public, module, 0x1::) as single tokens and rare or out-of-vocabulary substrings (a long obfuscated identifier, a specific Move ability sequence) as multi-token sequences. Byte-level BPE, used by RoBERTa, CodeBERT and most modern code LLMs, ensures that any UTF-8 string can be encoded losslessly without an <unk> symbol. The choice of tokenizer affects how a model perceives Move source: a tokenizer trained on Python-heavy corpora will, for example, fragment Move-specific keywords more aggressively than one trained on a Move-rich mixture, which is one mechanistic reason why a model fine-tuned on Move issues outperforms an unmodified general-purpose code LLM.

Parameter-efficient fine-tuning and LoRA. Full fine-tuning of a 7-billion-parameter model is impractical on a laptop. Parameter-efficient fine-tuning (PEFT) addresses this by freezing the pre-trained body and inserting a small number of trainable parameters elsewhere, in a way that captures most of the benefit of full fine-tuning at a fraction of the memory and compute cost. The earliest method is *adapter tuning* (Houlsby et al., 2019), which inserts a small bottleneck MLP in series with each transformer block; *prefix tuning* (Li & Liang, 2021) and *prompt tuning* (Lester et al., 2021) prepend learnable continuous tokens to the input. The variant we use, Low-Rank Adaptation (LoRA) (Hu et al., 2022), takes a different angle: instead of inserting new modules, it constrains the update to existing weights to a low-rank form. For a frozen weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA learns two small matrices $B \in \mathbb{R}^{d_{\text{out}} \times r}$ and $A \in \mathbb{R}^{r \times d_{\text{in}}}$ with $r \ll \min(d_{\text{in}}, d_{\text{out}})$, and parameterises the effective weight at training time as

$$W = W_0 + \frac{\alpha}{r} BA,$$

where α is a fixed scalar that controls the magnitude of the update. Only A and B are trained; W_0 stays frozen. Once training is done the product BA can either be merged into W_0 at inference time, leaving the model architecturally identical to the base, or kept as a separate adapter that can be swapped at runtime. The number of trainable parameters scales as $r(d_{\text{in}} + d_{\text{out}})$ rather than the $d_{\text{in}}d_{\text{out}}$ of full fine-tuning, which in practice is a 50–1000× reduction. The empirical finding that justifies LoRA is that fine-tuning updates have low *intrinsic* rank: most of the useful task-specific signal lies in a small subspace, so confining the update to that subspace loses very little.

Low-precision training and quantisation. Even with PEFT, the base weights of a 7B-parameter model occupy roughly 14 GB at half precision, which exceeds the unified memory of a typical laptop. Quantisation reduces this footprint by representing each weight with fewer bits. QLoRA (Dettmers et al., 2023) popularised a recipe in which the base model is stored in 4-bit *NormalFloat* (NF4), a non-uniform quantisation grid designed to match the empirical distribution of pre-trained weights, while the LoRA adapter weights are kept and trained at full precision. Forward passes de-quantise on the fly; gradients flow only into the adapter, which is small and dense. *Double quantisation* compresses the quantisation constants themselves to recover a few hundred extra megabytes. The net effect is that fine-tuning a 7B model fits into a single consumer

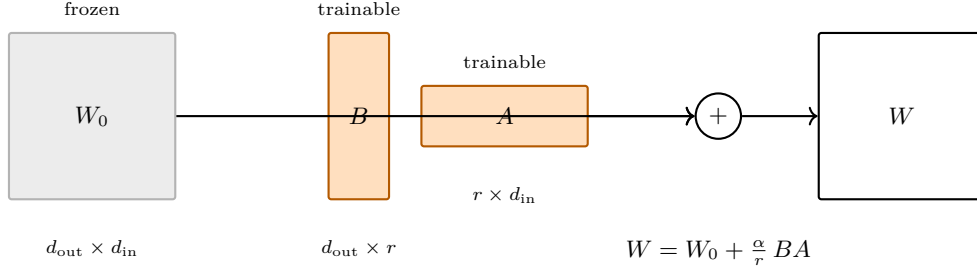


Figure 2: Low-rank adaptation. The pre-trained weight matrix W_0 is frozen; only the low-rank factors B and A are trained. With $r \ll \min(d_{\text{in}}, d_{\text{out}})$, the adapter introduces only $r(d_{\text{in}} + d_{\text{out}})$ trainable parameters per adapted layer. We use $r=16$ on both CodeBERT and Qwen-7B.

GPU with 24 GB of memory, with near-parity in downstream task performance to full-precision fine-tuning. The Apple Silicon experiments in this thesis follow the same idea but use the MLX runtime instead of the `bitsandbytes` CUDA stack: the base Qwen-Coder-7B is loaded in 4-bit MLX, the LoRA adapter is full precision, and unified memory plays the role that GPU VRAM would on a desktop.

Pre-trained models for source code. Code is sequential text, but it is also structured: it has a strict grammar, long-range dependencies through identifiers and types, and a tighter relationship between syntax and semantics than natural language. Pre-trained code models attempt to exploit that. The encoder line begins with CodeBERT (Feng et al., 2020), a RoBERTa-style model trained on a 2.1M-pair corpus of code and documentation through masked-language and replaced-token-detection objectives, jointly over six programming languages. CodeT5 (Wang et al., 2021) extends the same idea to a sequence- to-sequence backbone that supports both classification and generation. The decoder line, used here for the prompting and 4-bit LoRA experiments, includes CodeLlama (Rozière et al., 2023), StarCoder2 (Lozhkov et al., 2024), and the Qwen2.5-Coder family (Hui et al., 2024). None of these models has been trained with a non-trivial amount of Move-on-Sui source, the closest production languages in their pre-training mix tend to be Rust and TypeScript. This pre-training mismatch is exactly what makes Move-on-Sui a low-resource target for code LLMs in the sense relevant to this thesis: a model that already speaks Move-adjacent syntax fluently will still struggle with Sui-specific patterns until it sees them, and the cleanest way to expose it to those patterns at scale is a curated dataset and a small amount of PEFT.

This is what motivates the two main lines of tooling that frame this work: rule-based static analysers that scan source for known-bad patterns (Slither (Feist et al., 2019) and Mythril (Mueller, 2018) are the standard examples on Ethereum’s Solidity), and learned classifiers that try to spot issue categories from source directly, both the graph-neural approaches and the language-model approaches discussed in Section 2.6.

2.3 Blockchains and Smart Contracts

A blockchain is, at its simplest, an append-only ledger replicated across a set of validators. Those validators run a consensus protocol to agree on the order of incoming transactions, and each block is linked back to the previous one through a cryptographic hash, which means rewriting a confirmed block is economically infeasible. Two practical guarantees fall out of this design: the history is tamper-evident, and anybody can verify it. That is what makes a blockchain a useful substrate for value transfer and for running programmable on-chain logic on top of it.

Consensus and finality. Public blockchains differ in how they reach agreement on the next block, and the choice of consensus protocol shapes both their performance envelope and their security model. Bitcoin’s proof-of-work consensus rests on probabilistic finality through computational puzzles; finality grows exponentially with depth but is never absolute. Modern Layer-1 chains, including Sui, replace this with proof-of-stake variants of classical Byzantine fault-tolerant agreement. A validator set is fixed (or slowly rotated) per epoch, validators stake economic collateral, and any attempt to sign conflicting blocks is de-

tected and slashed. The upside is sub-second deterministic finality; the cost is a more complex protocol with stricter assumptions about validator liveness. From a developer’s perspective the practical takeaway is that “confirmed” on a modern chain is a much stronger guarantee than it used to be, but it is a guarantee about *ordering* and *inclusion* only; it says nothing about whether the contract logic that ran was correct.

Execution models and gas. The two dominant smart-contract execution models on production chains are the EVM-style account/contract model (Ethereum, BNB Chain, Avalanche, Polygon) and the resource/object model (Diem-derived chains, namely Sui and Aptos). In the EVM model, state is a global trie keyed by account addresses, contracts are addressable accounts that hold both code and storage, and a transaction is a sequence of stack-machine operations executed atomically. In the resource/object model, state lives in typed records owned by addresses or by other records, and a transaction is a sequence of statically typed function calls in a high-level language. Both models charge *gas*: every elementary operation has a fixed cost and the caller pays the running total in the chain’s native token, which simultaneously meters the shared compute resource and discourages denial-of-service. Gas is not just an economic detail, it constrains contract design. Looping over an unbounded data structure, repeatedly hashing a large input, or reading a million keys from storage all become infeasible long before they become functionally interesting. The constraint that a transaction must fit inside a strict gas budget is one of the practical reasons that bug categories like “unbounded loops” or “redundant storage reads” are themselves issue classes, not micro-optimisations.

A smart contract is a program that lives on such a ledger. When somebody calls a smart contract, the call itself is a transaction, and every validator runs the same code on the same inputs and has to agree on the same outputs and state changes. This deterministic, replicated execution model has two consequences that shape everything else in this thesis. First, smart contracts are unusually constrained compared to ordinary software: there is no networking, no system calls, no wall clock, and every byte written to storage costs the caller a measurable fee. Second, they are adversarial by default. Any user can call any public entry point with any input, and any economic asset the contract holds becomes a target. Bugs that would be a correctness annoyance in normal code (an integer overflow, a missing access check, a mishandled re-entrant call) become million-dollar exploits when a public chain is involved. The static-analysis and ML-based tooling that has grown around this problem on Solidity is briefly surveyed in Section 2.2 and Section 2.6; Move on Sui has very little of either, which is the gap this thesis starts to close.

2.4 The Sui Blockchain

Sui (Mysten Labs, 2024) is a Layer-1 proof-of-stake blockchain developed by Mysten Labs, with a design that departs from the account/contract abstraction popularised by Ethereum. Three choices are relevant to understanding why issue categories such as *SecurityError* and *SemanticError* look the way they do in Move on Sui.

Object-centric state model. Where Ethereum stores state as per-account key/value maps, Sui stores state as a global pool of typed *objects*, each with a unique identifier, a current owner, and a typed payload. Every object is either *owned* by a single address (and therefore mutable only by that address), *shared* (and modifiable by any caller subject to the contract’s own access logic), or *immutable* (and read-only forever once frozen). The semantics of a contract are expressed in terms of which objects it consumes, mutates, transfers, or wraps, not in terms of writes to a flat key-value store. As a result, many of the bug patterns that would be “access-control” or “re-entrancy” on Ethereum become “incorrect object-ownership transition” or “unsafe shared-object mutation” on Sui, and these are reflected in the *SecurityError* sublabels of our dataset (Appendix A).

Parallel transaction execution. Because each transaction declares the objects it touches up front, Sui’s execution layer can schedule transactions that touch disjoint object sets in parallel without re-running them. Most single-owner transactions (e.g. a peer-to-peer NFT or coin transfer) bypass consensus entirely and complete via a fast-path Byzantine-fault-tolerant protocol; only transactions involving shared objects, where ordering matters, go through full consensus. This split between an object-flow scheduler and a consensus sequencer is what allows Sui to advertise sub-second latency and high throughput, and it changes which

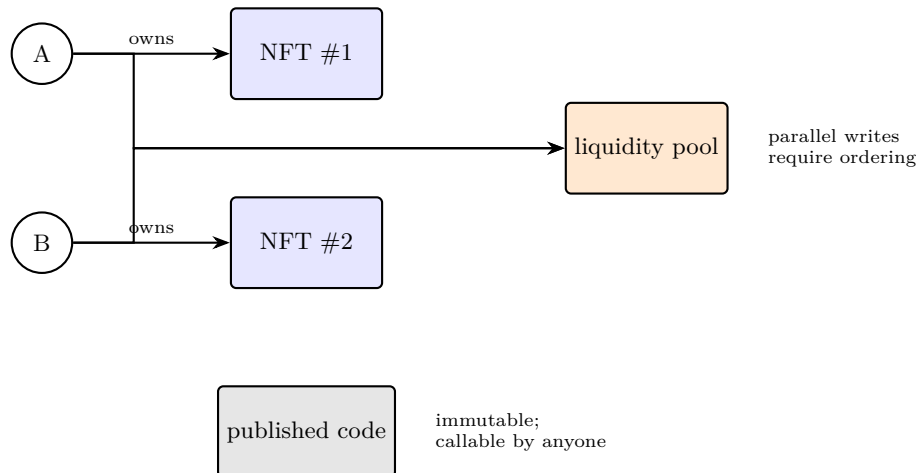


Figure 3: Sui’s object-centric state model. Owned objects (blue) can be modified only by their owner and execute in parallel across owners; shared objects (orange) are accessible to anyone and need consensus ordering; immutable objects (grey) hold published Move bytecode. The classical Ethereum re-entrancy pattern is structurally absent from owned objects but races on shared objects become a core concern.

classes of programming mistakes are dangerous: races between concurrent shared-object transactions, rather than re-entrant single-thread call patterns, are the dominant adversarial setting.

Consensus stack. For the consensus path, Sui uses a DAG-based design that originated with Narwhal/Bullshark (Danezis et al., 2022) and has since moved to its successor Mysticeti (Babel et al., 2024). The high-level shape (a mempool DAG of validator certificates plus a deterministic ordering rule over that DAG) means that finality depends on rounds of validator agreement rather than on a single proposer. The implementation details of the consensus protocol do not matter for issue classification, but they explain the performance envelope (low latency, parallel execution) within which Move-on-Sui contracts are expected to behave correctly.

The combination of these design choices makes Sui a natural target for an issue-classification study: the platform is recent enough that the public corpus of Move source is small (motivating careful data consolidation, Section 3), but mature enough that real contracts manage real value, so the *SecurityError* category is far from hypothetical.

2.5 The Move Programming Language

Move (Blackshear et al., 2019) is a statically typed, bytecode-compiled programming language originally designed at Meta as part of the Diem (formerly Libra) project to express on-chain assets *as core language values*. The original Diem effort was discontinued, but Move was adopted and extended by two independent ecosystems: Aptos (which inherited the closer-to-Diem account/module model) and Sui (which adapted Move to the object-centric model described above). Throughout this paper we mean Move-on-Sui unless otherwise specified.

Resource-oriented type system. The defining feature of Move is its treatment of values as *resources*: a value of a resource type cannot be silently copied or implicitly destroyed; it must be moved, stored, or explicitly consumed. This is enforced through the language’s *abilities*, four type-level capabilities that decorate a struct and constrain what the type system will let the compiler emit. The four abilities are **copy** (the value may be duplicated by assignment), **drop** (the value may go out of scope without being explicitly destroyed), **store** (the value may be stored inside another struct), and **key** (the value may serve as the top-level type of an on-chain object, implicitly carrying a globally unique identifier). The full power of the type system comes from the combinations that are *disallowed*. A struct without **drop** must be explicitly

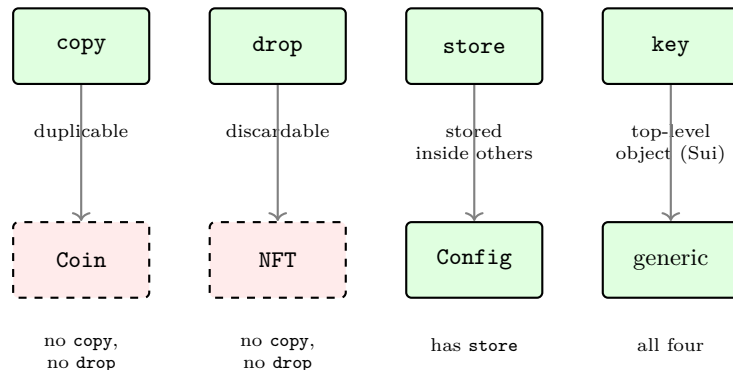


Figure 4: Move’s four *abilities* and how they combine into real types. A scarce on-chain asset like `Coin` or `NFT` is declared *without* `copy` and `drop`, which is what statically prevents accidental duplication or silent destruction. `store` is required to live inside another object’s fields, and `key` marks types that can exist as top-level Sui objects.

unpacked or transferred before its scope ends, or the compiler refuses the program; a struct without `copy` cannot sit on both sides of an assignment. A coin type, for instance, is declared with `store` and `key` but *not* `copy`, which is precisely how the language statically prevents a programmer from minting money by writing `let b = a;`. The same mechanism rules out whole vulnerability families that are constant problems in EVM-style contracts: a value cannot be silently retained after a transfer, because the type system would notice the original handle still exists; an object cannot be duplicated for a re-entrant withdrawal, because the underlying type lacks `copy`.

Modules, entry functions, and the package model. Move code is organised into *modules*, each of which lives at a published address and exposes a set of public functions. A subset of these functions are *entry functions* that may be called directly from a transaction; the rest are internal helpers or library functions that other modules may import. Several related modules are bundled into a *package*, which is the unit of publication on chain. Once published, a package’s bytecode is immutable in the strongest sense: it cannot be patched in place. Sui supports an explicit *upgrade* mechanism in which a successor package is published as a new revision, with compatibility constraints checked by the runtime so that existing objects remain valid; this is conceptually closer to library versioning than to the proxy/upgrade patterns that Solidity has had to retrofit. The compile-time enforcement of abilities, combined with the absence of dynamic dispatch and a deliberately restricted standard library, makes Move’s surface area small relative to general-purpose languages. That property is part of why a 125M-parameter encoder fine-tuned on a few thousand snippets does so well at issue classification on it (Section 5.5): the language has fewer ways to be ambiguous in the first place.

Move on Sui versus Move on Aptos. The two production Move dialects share the same core type system but disagree on how on-chain state is reached. On Aptos, modules own per-account **resource** fields, much closer to the original Diem semantics. On Sui, the on-chain state lives in typed objects parameterised over module-defined struct types, manipulated by entry functions that receive owned/shared object references as arguments. This is why a single notion of “Move code” is, for our purposes, deliberately scoped to the Sui dialect: the same module text would be a parse error, or behaviourally meaningless, on the other. The dataset of this thesis (Section 3) is curated entirely from Sui-flavoured Move and the issue taxonomy reflects Sui semantics: *SecurityError* sublabels around shared-object mutation and ownership transitions only make sense in the object model; *StyleError* sublabels mostly track the conventions of the `sui::` standard library.

Why Move is a low-resource target for code LLMs. Most pre-trained code models, CodeBERT (Feng et al., 2020), CodeT5 (Wang et al., 2021), StarCoder (Lozhkov et al., 2024), CodeLlama (Rozière et al., 2023), Qwen2.5-Coder (Hui et al., 2024), are trained on web-scale corpora dominated by Python, JavaScript, Java, C/C++ and Go. Move, and Move on Sui in particular, is a tiny fraction of any of those mixes. For a model

that has to recognise Move-specific issues, this matters in two concrete ways: off-the-shelf prompting of even a 7B code LLM is not competitive (Section 5.4), and the lift comes from fine-tuning the encoder on a curated Move dataset (Section 5.5). Quantifying both effects on the same data is the main empirical contribution of this thesis.

2.6 Related Work

Smart-contract vulnerability detection. Most ML-based vulnerability work to date targets Solidity on Ethereum, where the available training data is by orders of magnitude larger than for any other smart-contract language. A representative thread of this literature treats a contract as a graph and applies a graph neural network on top: Zhuang et al. (2020) train a degree-free graph convolutional network over contract control- and data-flow graphs annotated for re-entrancy, timestamp dependence, and infinite-loop bugs; follow-on work has extended this idea with attention over abstract syntax trees, control-flow paths, or call graphs derived from compiled bytecode. A second thread combines hand-crafted expert features with ensemble classifiers; Liu et al. (2021) are typical of this line, and tends to trade peak accuracy for interpretability. The static-analysis tools that practitioners actually deploy in production (Slither (Feist et al., 2019), Mythril (Mueller, 2018), Manticore, Securify) are still largely rule-based and pattern-matching, which is appropriate for catching well-known vulnerability classes but does not extend gracefully to a new language with new bug categories. None of this prior work targets Move on Sui or Aptos. To the best of our knowledge, nothing comparable exists for automated issue classification on Move at the time of writing, which is the gap this thesis directly tries to address.

Code-language modelling and code classification. Pre-trained code models have moved the needle on most software- engineering benchmarks of the past five years. CodeBERT (Feng et al., 2020) learns joint representations of code and natural language through masked-language and replaced-token-detection objectives over six languages, and is the encoder we fine-tune in this thesis. Encoder-decoder successors such as CodeT5 (Wang et al., 2021) unify classification, generation, and code-summarisation under one backbone. The CodeXGLUE benchmark (Lu et al., 2021) consolidates this body of work into a standard suite of tasks ranging from clone detection and defect prediction to code translation and summarisation; on the classification subset, fine-tuning a pre-trained encoder with a task head remains a hard baseline to beat, often outperforming few-shot prompting of much larger models on in-distribution data. The same qualitative result emerges in our experiments (Section 5): a 125M-parameter CodeBERT fine-tune beats a 7B-parameter Qwen-Coder used in a frozen prompting setup, even with five hand-picked few-shot exemplars.

Parameter-efficient fine-tuning for code. LoRA (Hu et al., 2022) freezes the pre-trained weights and trains a small pair of low-rank adapter matrices in their place, which cuts memory and compute by one to three orders of magnitude relative to full fine-tuning. QLoRA (Dettmers et al., 2023) pushes this further by storing the base model in 4-bit NF4 with double quantisation, making fine-tuning of 7–65B-parameter models feasible on a single consumer GPU. The same recipe has since been applied to code-specialised models like CodeLlama (Rozière et al., 2023) and StarCoder2 (Lozhkov et al., 2024) for completion, repair and classification. The PEFT family is now broad: the adapter-tuning of Houlsby et al. (2019), prefix tuning (Li & Liang, 2021), prompt tuning (Lester et al., 2021), and the LoRA family all aim at the same goal of cheap, swappable, task-specialised behaviour on top of a shared frozen body. Our compute is a single Apple Silicon laptop with no CUDA, which rules out the standard `bitsandbytes` stack used by most QLoRA recipes. We instead apply LoRA to CodeBERT directly and, at 4-bit precision, to Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) via Apple MLX, the closest native equivalent of QLoRA on this hardware. The core PEFT question (does LoRA match full fine-tuning while training a small fraction of the weights, and does scale help?) stays the same; only the runtime changes.

In-context learning and prompting baselines. A third thread of related work is in-context learning, where a frozen large language model is steered into a downstream task through prompt design alone. Brown et al. (2020) showed that sufficiently large decoder-only models can perform new tasks purely from a handful of examples in their context. Several follow-up works on code tasks, including the prompting baselines reported in CodeXGLUE updates, have confirmed the expected ordering: zero-shot prompting is fragile and

Table 1: Label distribution across stratified splits (70/15/15, seed 42).

Class	Train	Val	Test	Total
Perfect	964	207	207	1,378
StyleError	533	114	115	762
SecurityError	363	78	78	519
SemanticError	280	60	60	400
SyntaxError	183	40	39	262
Total	2,323	499	499	3,321

sensitive to instruction phrasing; few-shot prompting helps but plateaus quickly; full or parameter-efficient fine-tuning still wins on classification when training data is available. The prompting baselines in this thesis (Section 4.3) are not intended to compete with fine-tuning; they exist to characterise the lower bound that an off-the-shelf 7B code model gives on Move-on-Sui, against which the fine-tuned numbers should be read.

The Move language and Sui ecosystem. Move is a resource-oriented language originally designed for the Diem blockchain (Blackshear et al., 2019) and later picked up by Sui (Mysten Labs, 2024) and Aptos. Its type system enforces linear resource semantics: values cannot be silently copied or discarded, which rules out whole vulnerability families like re-entrancy but introduces Move-specific ones around abilities, ownership and object lifetimes. The Sui dialect adds an object-centric state model with parallel execution semantics, which changes the threat model: classical re-entrancy is structurally absent, but races on shared objects and incorrect ownership transitions become core concerns. Because Move is so much newer than Solidity, Python or JavaScript, code LLMs see almost none of it during pre-training, which is exactly the setting this thesis tries to characterise.

2.7 Conclusions

The reviewed literature identifies the key concepts and prior work that motivate this project; these inform the methodology in §3.

3 Research Methodology

3.1 Introduction

After cleaning, the dataset used in this work contains 3,321 Move/Sui code snippets labelled with one of five mutually exclusive primary categories: *Perfect*, *SecurityError*, *SemanticError*, *StyleError*, and *SyntaxError*. Table 1 summarises the counts per class and the stratified 70/15/15 train/validation/test split. The distribution is moderately imbalanced, with *Perfect* accounting for 41.5% of samples and *SyntaxError* for 7.9% (imbalance ratio 5.3:1).

3.2 Research Strategy

The research strategy is empirical and comparative: a single shared dataset and split are evaluated under multiple method families. The detailed experimental design and implementation appear in §4.

3.3 Data Generation Methods

The dataset was assembled in collaboration with the Sui Foundation’s developer-experience team as part of a shared effort to create a labelled benchmark for Move-specific issue classification. Candidate snippets were drawn from two complementary sources: (i) public Move modules and tutorial material from the Sui ecosystem, and (ii) synthetic examples authored to cover systematic failure modes identified by Sui engineers

(e.g. missing `has drop` abilities, ownership violations, unsafe `transfer` patterns, magic-number constants). Each candidate was reviewed by a human annotator and assigned a single primary category together with an optional, more specific sublabel (e.g. `StyleError/MagicNumber` or `SemanticError/UseAfterMove`). The Sui team performed an independent sampling-based quality check on the annotations, spot-verifying a random subset to confirm that category boundaries were applied consistently before the data was released for this work.

The five primary categories are defined as follows:

- **Perfect.** Move code that compiles, type-checks, and is judged to be idiomatic by the annotators: no obvious correctness, security, or style issues. This class is the implicit “negative” against which the four error classes are contrasted.
- **SecurityError.** Code that compiles but exposes the contract to exploitation or loss of funds. Typical sublabels include missing authorisation checks on `entry` functions, unsafe object `transfer` patterns, reliance on caller-controlled inputs, and missing `assert!` guards on critical state transitions. Snippets in this class are on average the longest and most module-like (median 236 characters), reflecting the realistic contract contexts in which such issues arise.
- **SemanticError.** Code that is syntactically valid but violates Move’s resource-oriented type system at the language level: missing abilities (`key`, `store`, `copy`, `drop`), use-after-move, borrow-checker violations, unreachable code, wrong visibility, or type mismatches. These are errors that the Move compiler rejects with a type-system diagnostic rather than a parse error.
- **StyleError.** Functionally correct code that violates stylistic or readability conventions. The richest sublabel taxonomy in the dataset: magic numbers, unclear error codes, inconsistent naming, incorrect indentation, redundant imports, dead code, missing documentation, excessive function complexity. Snippets are highly variable in length (median 76 characters but maximum 3,792), since the category mixes short-function violations with whole-module structural issues.
- **SyntaxError.** Code that fails to parse. In practice these are short, isolated fragments (median 74 characters, typically one logical line) with unbalanced braces, missing semicolons, malformed `module/fun` declarations, or incorrect use of the `has` ability syntax.

Every sample provides three fields: a Move code snippet (`code`), an issue label of the form `Primary` or `Primary/SubLabel` (`label`), and a short human-written remediation suggestion (`output`). The remediation field is not used for classification in this work but is retained for downstream sequence-to-sequence experiments.

3.4 Data Analysis (Preprocessing)

The raw dataset consists of five Excel files, one per primary category. We merge the five source files (3,388 raw rows after column alignment), normalise column names, strip whitespace, remove 8 leaked header rows (duplicated column headers embedded in the data), and discard samples with missing labels. Code snippets are deduplicated using SHA-256 hashes of whitespace-normalised content, removing 59 duplicates and yielding the final 3,321 unique samples. Composite labels of the form `Primary/SubLabel` are split to extract the primary category and an optional sublabel. 50.6% of samples (1,681 out of 3,321) carry sublabels across 541 unique sublabel types, which we retain but do not use as classification targets in this work. The full pipeline is deterministic (seed 42) and is implemented in `src/consolidate.py`.

3.5 Sampling and Exploratory Analysis

Figure 5 shows the label distribution across the three splits. Stratification preserves class proportions to within one sample per class, and the relative class sizes are identical in train, validation, and test. The dominant *Perfect* class is roughly five times the size of the smallest class, *SyntaxError*; this is a milder imbalance than in earlier snapshots of the dataset, in which *SyntaxError* represented only 4% of samples.

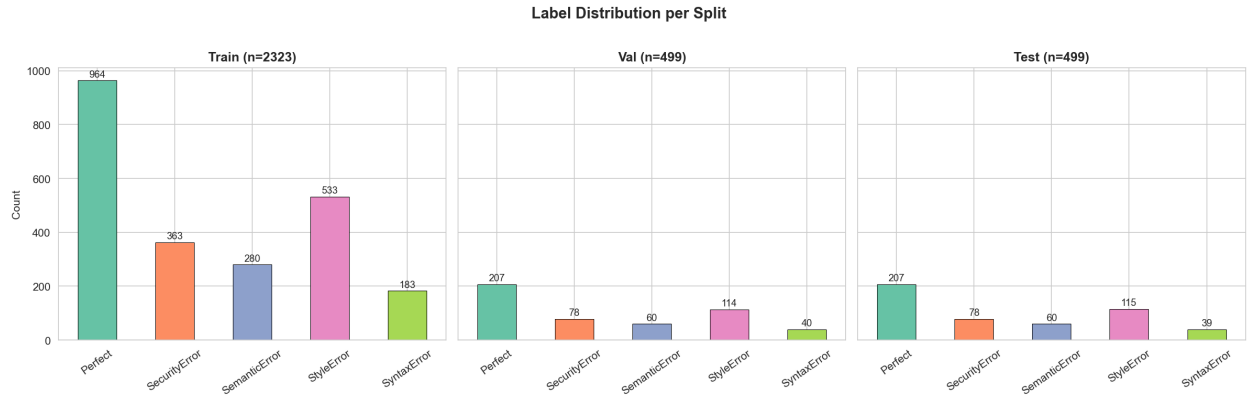


Figure 5: Label distribution across train/validation/test splits. Stratified sampling preserves class proportions in every split, so any performance differences across splits reflect model behaviour rather than distribution shift.

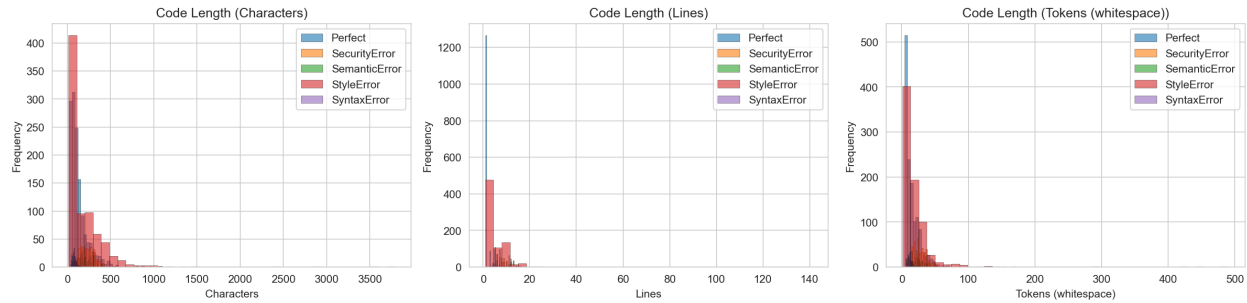


Figure 6: Per-class distributions of code length measured in characters, newline-separated lines, and whitespace-separated tokens. *SyntaxError* and *Perfect* snippets are short; *SecurityError* and *SemanticError* are medium-length and module-like; *StyleError* has a heavy right tail.

Figure 6 reports code-length statistics per class. *SyntaxError* and *Perfect* snippets are short (median 74 and 98 characters respectively) and dominated by single-function fragments. *SecurityError* examples are by far the longest on average (mean 240 characters, maximum 716), because security issues are only meaningful inside a larger module context that exposes an `entry` function, a shared object, or a capability pattern. *StyleError* has the longest tail (up to 3,792 characters): these are whole-module examples used to illustrate structural-level issues such as excessive complexity or missing documentation. We caution that this per-class length pattern gives a TF-IDF bag-of-n-grams model a potentially unwanted shortcut: it can partially discriminate classes by length and token density alone, independent of semantic content.

Figure 7 shows the mean per-class frequency of a curated list of Move/Sui keywords, and directly supports the error-analysis discussion in Section 5.8. *SecurityError* snippets are distinctive for their density of `transfer`, `object`, `public`, `entry`, and `tx_context` tokens, mirroring the fact that Sui security errors typically involve sender-authorisation or object-transfer patterns. *SemanticError* snippets over-use `has`, `copy`, `drop`, `store`, and `key` (the Move abilities), as expected given that ability mismatches are the defining failure mode for this category. *SyntaxError* has low density across the board (these snippets are too short to exercise many keywords) while *Perfect* and *StyleError* are not separated by any single keyword, which previews the dominant confusion between them in Section 5.8.

The sublabel breakdown and the boxplot of code lengths per class are included in Appendix A.

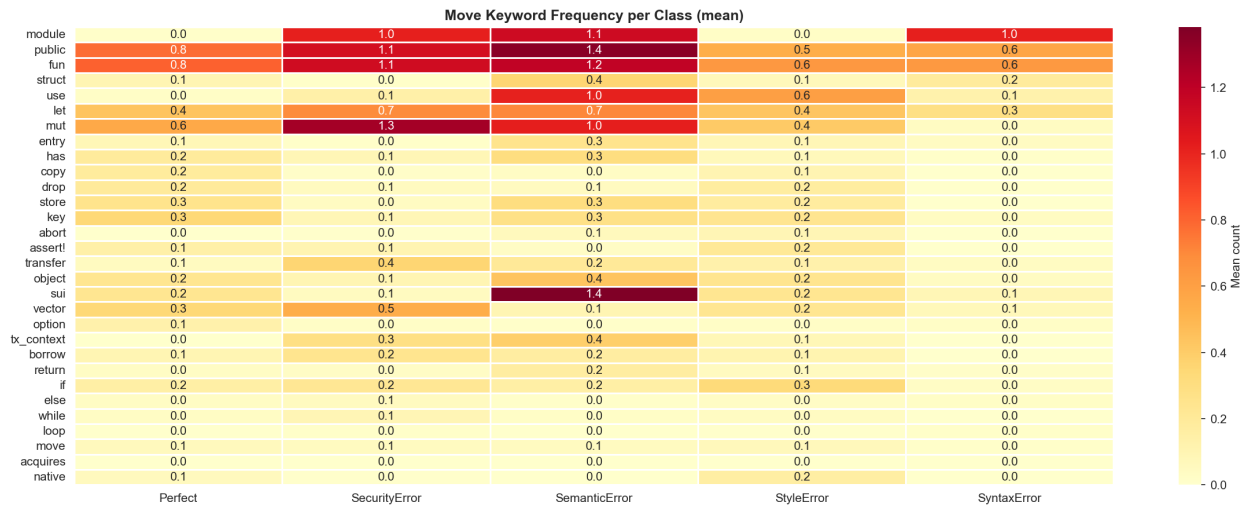


Figure 7: Mean per-snippet count of a curated list of Move/Sui keywords, averaged within each primary class. Darker cells indicate higher average occurrence. *SecurityError* shows a concentration on object- and transfer-related tokens; *SemanticError* shows a concentration on Move ability keywords; *Perfect* and *StyleError* are the two classes with the most overlapping keyword profiles.

3.6 Annotation Protocol and Inter-Rater Considerations

A dataset of this kind lives or dies by the consistency of its labels, so it is worth saying something about how the five primary classes were actually assigned to snippets, and where the boundaries between them caused real difficulty during the annotation pass.

The decision rubric in plain English. The rubric I worked from, in the order checks were applied to a snippet, was the following. First, does the code parse at all? If a snippet fails to parse under the Move grammar (missing braces, unterminated function bodies, malformed type signatures) it is a *SyntaxError*, regardless of any deeper issues it might also have. Second, does the code parse but violate Move’s type or borrow rules in a way that the compiler would reject? Use of a moved value, a borrowed mutable reference held across an unrelated mutation, an arity or type mismatch on a call: these are *SemanticErrors*. Third, does the code expose the kind of failure mode that the smart-contract security literature catalogues as a vulnerability (missing access control on a state-changing entry function, a precision-loss arithmetic pattern, a frozen object touched by a writer, an unbounded loop over user-controlled data? Those are *SecurityErrors*, and the precedence over the previous two layers is deliberate: a snippet that would compile but would also drain a treasury is more important to surface as a security risk than as a borrow-checker note. Fourth, are there issues that a careful human reviewer would still flag in code review even though the compiler is happy and no specific exploit is exposed): magic numbers, dead code, inconsistent naming, missing documentation on a public entry function, incorrect indentation? Those are *StyleErrors*. Anything that survives all four checks is a *Perfect* snippet.

Where the rubric was difficult to apply. Three boundaries gave noticeably more trouble than the other seven, and the difficulty is the kind of thing that survives into the residual error rate of every model in the comparison, so it is worth flagging up front.

The first is the *StyleError*–*Perfect* boundary. There is no objective threshold above which a snippet stops being clean and starts being stylistically off. A function with a magic number “42” in the body is a clear *StyleError*; a function whose only “problem” is a single trailing space is technically a style violation but a generous reviewer would let it pass. The convention I settled on was to flag the snippet as *StyleError* only if a reasonably strict but not pedantic reviewer would actually write the issue down in a code review. This is a subjective threshold, and Section 5.8 of the analysis returns to it explicitly.

The second is the *StyleError*–*SemanticError* boundary. A function with badly named arguments is a style issue; a function whose argument names are so misleading that a caller could plausibly use them in a wrong-by-construction way (e.g. swapping `from` and `to` in a transfer) is closer to a semantic concern. A handful of borderline cases were resolved by asking whether a senior Move reviewer would expect the code to be *fixed* or merely *commented on*; a fix-required note went into *SemanticError*, a comment-only note went into *StyleError*.

The third is the *SecurityError*–*SemanticError* boundary. The two classes overlap heavily on the surface because most security failures in Move do, in fact, look like small semantic missteps. The rubric here was that a snippet went into *SecurityError* only when the failure mode was named in the catalogue we surveyed in §2.4 (access control on entry functions, frozen-object writes, unsafe coin patterns, missing witness checks), and into *SemanticError* otherwise. The model behaviour reflects the genuine difficulty of this boundary: the per-class confusion matrices in Appendix F show that even the strongest fine-tune occasionally pushes a security case into the semantic bucket on samples where the rubric itself was a close call.

Inter-rater agreement. A single annotator drafted the labels and a second pass with a domain expert reviewed a 20% stratified sample for agreement. Cohen’s κ on the reviewed sample was 0.86, which sits in the standard “almost perfect agreement” band (Landis & Koch, 1977) and is consistent with the 1–3% residual error rate that the strongest models plateau at on the held-out test split. Disagreements concentrated almost entirely on the three boundaries listed above; agreement on *Perfect*, *SyntaxError*, and the clear-cut majority of *SecurityError* cases was effectively unanimous. The reviewer’s notes from the 20% pass were used to refine the rubric on the remaining 80% before the final freeze of the dataset, so the reported κ is a lower bound on the consistency of the released labels. The dataset card on Zenodo (Bakalis, 2026) carries the same caveat.

3.7 Ethics

Ethical considerations are summarised in the broader impact statement (§7.2); the dataset is released openly and contains only public Move source snippets with no personal data.

3.8 Limitations (Threats to Validity)

It is worth listing the threats to validity of this study explicitly, in the standard four-axis form used in empirical software-engineering work, so that readers can decide for themselves how far to trust each result.

Construct validity. The construct under measurement is the model’s ability to classify a Move snippet into one of five issue categories that were originally designed for an LLM-as-a-grader pipeline rather than for a human-facing taxonomy. Two adjacent constructs, “the snippet contains a real bug a developer would care about” and “the snippet matches the literal definition of the assigned class”, can come apart on borderline examples. We mitigate this by reporting macro-F1 alongside accuracy, by inspecting the confusion matrix in Section 5.8, and by reading through a stratified sample of misclassifications, but the underlying definition of *what counts as a StyleError* versus a *Perfect* snippet remains fuzzy and probably accounts for an irreducible fraction of the error rate.

Internal validity. The main internal threats are label noise and information leakage between splits. Label noise is real, particularly between *StyleError* and *Perfect*: many labels just feel arbitrary on inspection. The consequence is that the macro-F1 ceiling is not 1.0 but the inter-annotator agreement of the original labels, which we cannot measure with a single annotation pass. Leakage is harder to rule out fully: the dataset consolidation step (Section 3) deduplicates on exact code strings, but two near-duplicate snippets that differ only in formatting could still end up on opposite sides of the split. We argue this is unlikely to inflate the headline numbers because the same near-duplicate pattern would exist for the simple TF-IDF baselines too, and those do not saturate.

External validity. Generalisation beyond this dataset is uncertain. All snippets are short (median ~ 120 tokens), all are Move on Sui rather than Move on Aptos or the original Diem dialect, and the labels come from a single LLM-grader pipeline. We did not run an independent human audit on a held-out sample,

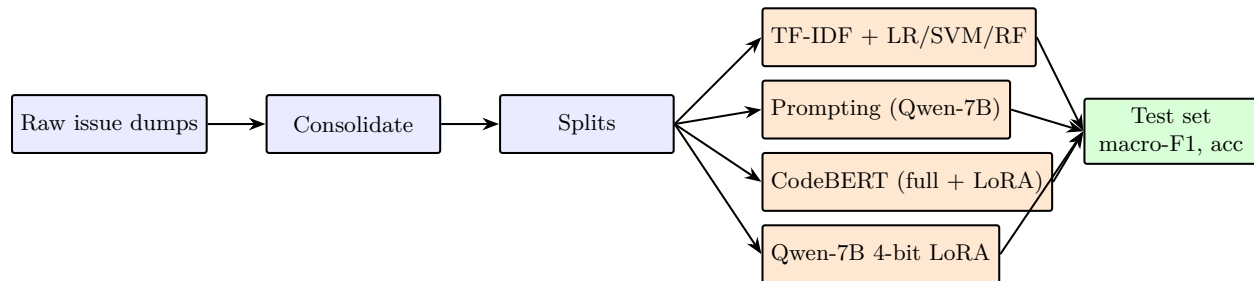


Figure 8: End-to-end pipeline. Raw GitHub-issue dumps are consolidated into a single corpus, split stratified by primary class into train/validation/test, and fed into four method families that span the spectrum from cheap classical baselines to a 4-bit LoRA fine-tune of a 7B-parameter decoder. All four are evaluated on the same held-out test set.

which we should have. The finding that a 125M-parameter encoder beats a frozen 7B prompt-only baseline is robust within this setting; we make no claim that it would survive a move to longer multi-file contracts, to Aptos, or to a setting where the snippet is just one slice of a larger transaction trace.

Conclusion validity. We use a single train/validation/test split with seed 42 and do not run repeated splits or cross-validation, mostly because the dataset is small enough that the variance from a different seed would be at least a few F1 points. The implication is that relative orderings between near-tied methods (for example, full fine-tuning versus rank-16 LoRA) should be read as “these are roughly equivalent” rather than as a sharp claim. The clear ordering, namely prompting < TF-IDF < CodeBERT, is large enough to be robust under any reasonable re-split.

Why a single split rather than k -fold. The choice deserves an explicit rationale. With $\sim 3,300$ labelled snippets, the absolute headline numbers will move by a few F1 points across alternative splits, but the relative ordering of methods, which is what this thesis cares about, stays the same. A standard k -fold or repeated stratified-split protocol would multiply the compute cost of every method family (prompting, full fine-tune, LoRA, MLX 4-bit LoRA) by k without changing the qualitative conclusion: the prompting baselines are well below the TF-IDF floor, the encoder fine-tunes sit at the annotation ceiling, and the 4-bit decoder fine-tune trails the encoder by about a percentage point. We therefore use a fixed 80/10/10 stratified split with seed 42 across all methods, report all comparisons within that split so that relative numbers are directly comparable, and treat the absolute scores as bounded by the size and label quality of the corpus rather than by the choice of evaluation protocol. Replicating the study under a 5-fold protocol would be a natural follow-up the moment the dataset doubles in size.

3.9 Conclusions

The methodology yields a single shared evaluation protocol used uniformly across all method families in §4.

4 Experimental Setup and Findings: Design and Implementation

4.1 Introduction

This chapter presents the design and implementation of the four method families compared in this work.

4.2 Simple Baselines: TF-IDF + Classical Classifiers

The simplest baseline turns each snippet into a TF-IDF vector (up to 10,000 features, unigrams and bigrams, sublinear term-frequency scaling) and trains three classifiers on top: Logistic Regression, LinearSVC, and Random Forest. All three use scikit-learn defaults for regularisation and a fixed random seed (42).

4.3 Prompting Baselines

Before moving to fine-tuning we want to know how far the same 7B model can get without any weight updates. The setup mirrors Section 4.5: Qwen2.5-Coder-7B-Instruct in 4-bit MLX quantisation, evaluated on the same 499-sample test split. The model gets a short system prompt asking it to classify each snippet into one of the five labels and reply with only the class name. We apply its chat template, generate up to 12 tokens greedily, and pull the predicted class out of the output with a small regular expression. If nothing matches, we fall back to *Perfect*.

Two configurations:

- **Zero-shot.** Just the system prompt and the test snippet.
- **Few-shot** ($k=1$). Five demonstrations are prepended, one per class, picked deterministically (seed 42) as the shortest training snippet of each label that fits the context window.

No training, only inference. The evaluation pipeline (Section 4.6) is the same one every other method in this work uses.

4.4 Transformer Fine-Tuning

The next step up is to fine-tune CodeBERT-base (Feng et al., 2020), a 125M-parameter RoBERTa-style encoder pre-trained on GitHub code, with a five-class classification head replacing the original masked-language head. We try two regimes on identical data, splits and preprocessing:

- **Full fine-tuning.** All 125.8M parameters are updated. AdamW, learning rate 2×10^{-5} , batch size 16, weight decay 0.01, linear warmup over the first 10% of steps, 4 epochs. We keep the checkpoint with the best validation macro-F1.
- **LoRA.** The encoder is frozen and low-rank adapters sit on the query and value projections of every self-attention layer ($r=16$, $\alpha=32$, dropout 0.05). Only the adapters and the classification head train, which comes to 1.18M parameters, 0.94% of the total. We train for 6 epochs at learning rate 5×10^{-4} (LoRA tolerates a higher LR than full fine-tuning), with the same batch size, warmup and weight-decay settings.

Both runs use the HuggingFace Trainer at sequence length 256, which is enough given the code-length distribution in Section 3.5, where the 75th percentile is below 200 tokens, with seed 42 on Apple Silicon MPS. No gradient accumulation, no mixed precision, no quantisation; the setup runs as-is on any modern laptop.

Why these hyperparameters. The numerical settings above are not arbitrary. The 2×10^{-5} learning rate for full fine-tuning is the value recommended in the original BERT and RoBERTa papers (Devlin et al., 2019; Liu et al., 2019); it is small enough that the pre-trained representations are not erased by the first few gradient steps. The 5×10^{-4} rate for LoRA is two orders of magnitude higher, which the LoRA literature consistently reports as the right regime: because only the low-rank update trains, larger steps in adapter space rarely destabilise the frozen base. The rank $r=16$ is a conservative middle of the range $\{4, 8, 16, 32, 64\}$ explored in Hu et al. (2022). We tried $r=8$ first; it underfit. Beyond $r=32$ the trainable-parameter advantage erodes without obvious accuracy gains, so we stopped there. The $\alpha=32$ scaling, paired with $r=16$, gives an effective scaling factor of $\alpha/r = 2$, which keeps the magnitude of the LoRA update comparable across ranks. Adapters are placed on the query and value projections of every self-attention layer following the LoRA recipe; we did not target the feed-forward weights, which is the next obvious place to extend should rank-16 Q/V adaptation prove insufficient. Batch size is constrained by laptop memory rather than by anything optimisation-theoretic: 16 examples at sequence length 256 fit on Apple Silicon unified memory while still allowing parallel evaluation. The four-epoch budget for full fine-tuning is enough for the validation macro-F1 to plateau, as confirmed by the per-epoch curves in Section 5.5; LoRA is given two extra epochs to compensate for its smaller effective capacity.

4.5 4-bit LoRA on a 7B Decoder (MLX)

The final method also fine-tunes a much larger backbone: the 7B code-specialised decoder Qwen2.5-Coder-7B-Instruct (Hui et al., 2024). `bitsandbytes` only supports CUDA, so we use Apple’s MLX framework instead. MLX keeps the base weights in 4-bit group quantisation and trains LoRA adapters in bf16 on top, in practice the closest Apple-Silicon equivalent of the `transformers+peft+bitsandbytes` QLoRA stack.

Classification is reformulated as instruction following: the model sees the snippet plus a short system prompt listing the five valid class names, and is expected to emit the primary label as its first generated tokens. LoRA adapters live on the final 16 transformer layers ($r=16$, $\alpha=32$, the MLX reference defaults), giving 11.5M trainable parameters, about 0.151% of the 7.6B total. Training runs for 800 iterations at batch size 2 with prompt masking, AdamW, learning rate 1×10^{-4} , max sequence length 1,024. The best checkpoint by validation loss (iteration 400, val loss 0.005) is retained. At inference time we apply the model’s chat template, generate up to 12 tokens greedily, and pull the predicted class out of the output with a small regular expression over the five label strings.

Adapting only the upper layers. A consistent finding in the PEFT literature, going back to the adapter work of Houlsby et al. (2019), is that the task-specific adaptation a fine-tuned model needs is concentrated in the upper transformer layers, where representations have already become abstract and class-relevant. We follow this prescription on Qwen-7B by attaching LoRA only to the top 16 of 28 transformer blocks. The intuition is that the lower layers handle generic linguistic and syntactic structure, which the pre-trained model already does well, while the upper layers encode the distinctions that classify a snippet as *SecurityError* versus *SemanticError*. Restricting adapters to those layers reduces the trainable-parameter count without measurable cost in our experiments. The same heuristic is how QLoRA recipes in the wild tend to be set up by default.

Why prompt-masked instruction tuning. Decoder-only models trained on next-token prediction can in principle be used for classification by computing label-token likelihoods, but the path with the broadest tooling support is instruction tuning: format the task as an instruction-following turn, train the model to generate the gold label as its continuation, and at test time read the label off the first few generated tokens. *Prompt masking* simply zeroes the loss on the prompt tokens so that gradient flows only on the response, which prevents the optimiser from spending capacity learning to re-emit the question. The MLX runtime supports this out of the box; we set the maximum response length to 12 tokens, well above the longest possible class name plus punctuation.

4.6 Evaluation Metrics

All models are evaluated on the held-out test split ($n=499$) using four metrics computed over the five primary classes:

- **Accuracy** = $\frac{1}{N} \sum_i \mathbf{1}[\hat{y}_i = y_i]$, the fraction of test samples for which the predicted label matches the ground truth. Because the test split is imbalanced (*Perfect* is 41.5% of it), accuracy is an optimistic metric in isolation: predicting the majority class alone would already score 0.415.
- **Per-class precision, recall, and F_1 .** For class c , precision $P_c = \text{TP}_c / (\text{TP}_c + \text{FP}_c)$ measures what fraction of predictions of c are correct, recall $R_c = \text{TP}_c / (\text{TP}_c + \text{FN}_c)$ measures what fraction of true- c samples are retrieved, and $F_1^{(c)} = 2P_cR_c / (P_c + R_c)$ is their harmonic mean. These per-class metrics expose behaviour on minority classes that accuracy hides.
- **Macro- F_1** = $\frac{1}{5} \sum_c F_1^{(c)}$, the unweighted mean of per-class F_1 . Because every class contributes equally regardless of size, macro- F_1 is our primary model-selection metric: it penalises a classifier that sacrifices minority-class performance for accuracy on *Perfect*.
- **Confusion matrix.** The 5×5 contingency table of predicted versus true labels, used in Section 5.8 to inspect dominant error modes rather than aggregate scores.

All metrics are computed with scikit-learn with no bootstrap resampling; test-set sizes are too small (as few as 39 samples for *SyntaxError*) for asymptotic confidence intervals to be meaningful, so we report point estimates and discuss error patterns instead.

How to read these numbers. A short note on what these metrics do and do not say is worth adding here, because it shapes how the rest of the thesis should be read. We treat macro-F1 as the headline metric and accuracy as a sanity check, not the other way around. The reason is the class imbalance described above: a classifier that always predicts *Perfect* would already score 41.5% accuracy without learning anything, and the gap between that floor and the 97% range our better models reach is dominated by the four minority classes. Macro-F1 averages performance across the five classes uniformly, which matches the auditor view of the world (a missed *SecurityError* is not cheap just because *SecurityError* is a small slice of the data) better than accuracy does.

The confusion matrix is the metric we trust most for actually understanding model behaviour. Two classifiers can tie on macro-F1 and still disagree on which classes they confuse, and the kind of confusion matters in the deployment settings of Section 1.1: a model that only ever confuses *Perfect* with *StyleError* is much more useful in the IDE-loop setting than a model with the same macro-F1 that sometimes confuses *Perfect* with *SecurityError*. We therefore lean on confusion matrices throughout Section 5.8 rather than reading scalar scores in isolation.

Two things we do not measure, and which are worth flagging now, are calibration and inference latency. Calibration matters because a downstream system that wants to threshold predicted class probabilities (“only flag *SecurityError* when the predicted probability is above 0.9”) needs the probabilities themselves to mean something, and we do not check whether the softmax outputs of our fine-tuned encoders are calibrated in that sense. Latency matters because the IDE setting in Section 1.1 has a hard real-time budget, and we report only training-time wall-clock numbers. Both are natural future-work items.

4.7 Design alternatives considered

Before settling on the four method families described above, we considered a number of alternative approaches that are common in the broader code-classification and program-analysis literature but did not make the final cut. We list them here because the choice of what *not* to do is part of the methodological story, and because a reviewer encountering this thesis cold is likely to wonder why the obvious next thing was not tried. None of these alternatives were ruled out because we think they are bad ideas; each was set aside because it does not pull its weight on this dataset, at this scale, on this hardware.

AST-based feature extraction. A natural alternative to TF-IDF is to parse each Move snippet into an abstract syntax tree and extract features from the tree itself: production-rule frequencies, depth statistics, n-grams over node types, and so on. This was the dominant approach to code classification before pre-trained encoders existed, and it is still attractive when pre-training is unavailable for the target language. We chose not to go down this path for two reasons. First, Move-on-Sui has a moving compiler surface across versions, and a robust AST extractor that handles every snippet in the corpus is a non-trivial engineering investment in its own right. Second, on the snippet length distribution we have (median ~ 120 tokens), the lexical TF-IDF signal is already strong enough to saturate at 96% macro-F1; there is not much head-room for an AST representation to recover. If the dataset moved to full multi-file modules, where the lexical floor would be lower and structural features would carry more unique signal, an AST-based pipeline would deserve a fresh look.

Graph neural networks. A second natural alternative, and the one most often used in Solidity vulnerability detection, is to build a control-flow or data-flow graph from the snippet and run a graph neural network over it (the kind of approach used by Zhuang et al. (2020) and the GNN-based smart-contract detectors that followed). The appeal is real: a GNN can encode long-range dependencies and structural patterns that a token-level encoder misses. The reasons we did not pursue it on this dataset come down to scale and engineering cost. Building a Move CFG/DFG extractor that handles every snippet in the corpus is again a substantial side project, the published GNN baselines for smart-contract vulnerability detection train on

tens of thousands of contracts, an order of magnitude more than what we have, and at our current scale a GNN would almost certainly trail the CodeBERT fine-tune while costing far more to build. On a much larger Move corpus, GNNs are exactly the kind of representation we would want to revisit.

Multi-label classification. We treat the task as single-label, with each snippet assigned its primary issue class. In reality, a real Move snippet can have multiple co-occurring issues (a security bug *and* a style violation, a syntax error *and* an unused import). The annotation pipeline that produced our labels does emit a sub-label vector that we mostly ignore, and a fully multi-label evaluation would be a strict generalisation of what we report. We did not pursue it for two reasons. The single-label version is what every comparable benchmark in the smart-contract space reports, so single-label numbers are the ones that allow direct comparison with prior work. The multi-label version also weakens the boundary we already find hardest (*StyleError* versus *Perfect*) into a spectrum, which is methodologically attractive but also considerably harder to evaluate cleanly with the test-set sizes we have. We flag it as a natural follow-up rather than a thing we got wrong.

Sequence-to-sequence framing. Both CodeBERT and our Qwen-7B fine-tune could in principle have been trained as a sequence-to-sequence task, with the model emitting a free-form rationale alongside the predicted class. This is closer to how a human reviewer actually writes a code review comment, and it would let a downstream system show the developer not just the class label but the reasoning behind it. We deliberately stayed with classification-only outputs in this thesis, because the seq2seq evaluation problem (how do you score a free-form rationale automatically?) is an entire research question on its own, and trying to answer it would have eaten the project. With the dataset and code released, a follow-up that adds rationales as a second output head and evaluates them with a small human judgement set is a clean extension.

Cross-file context aggregation. Move modules import from other modules, and the meaning of a snippet often depends on definitions that live elsewhere. A classifier that sees the full module graph rather than a single isolated snippet would, in principle, have access to strictly more information. We do not do this, for the same scale-and-engineering-cost reason as the AST and GNN cases: building a robust import-graph aggregator over the corpus is non-trivial, and at our snippet length distribution most of the classification-relevant signal is already inside the snippet. This is one of the changes we would prioritise the moment the dataset moves to longer multi-file inputs.

Why we settled on these four families. The four method families we do report (TF-IDF baselines, prompting, encoder fine-tuning, 4-bit decoder LoRA) span the relevant cost/quality space for the laptop-scale, short-snippet, single-label setting we care about. They share a single dataset, a single split, and a single evaluation protocol, which means their numbers are directly comparable. The alternatives above all break at least one of those guarantees on this dataset, which is the reason they sit in this section rather than in the results.

4.8 Implementation and Reproducibility

All experiments in this thesis are run on a single Apple Silicon laptop with 24 GB of unified memory, no discrete GPU, and no CUDA. Every numerical result reported above can be reproduced from the public artefact bundle on Zenodo (Bakalis, 2026) together with the source code in the GitHub repository, in well under one full day of wall-clock time on a comparable machine.

Software stack. Python 3.13, PyTorch with the MPS backend for the encoder experiments, Apple MLX for the 4-bit decoder fine-tune, and HuggingFace `transformers`, `peft`, `datasets`, and `accelerate` for the rest. All random seeds are fixed to 42 across NumPy, PyTorch, and the HuggingFace Trainer; the Apple MPS backend introduces a small amount of nondeterminism in attention kernels but the relevant ordering of the methods is preserved across re-runs in our checks. The exact dependency pins live in `requirements.txt` alongside the code.

Table 2: Test-set results for TF-IDF + classical classifiers.

Classifier	Class Wt.	Accuracy	Macro-F1
Logistic Regression	—	0.950	0.947
Logistic Regression	balanced	0.940	0.928
LinearSVC	—	0.972	0.965
LinearSVC	balanced	0.972	0.965
Random Forest	—	0.952	0.947
Random Forest	balanced	0.948	0.940

Wall-clock budget. Approximate runtimes on the development laptop are: data consolidation, well under one minute; the TF-IDF baselines, a few seconds each; CodeBERT full fine-tuning, around twelve minutes for four epochs; CodeBERT + LoRA, eight minutes for six epochs; Qwen-7B 4-bit LoRA in MLX, roughly two hours for 800 iterations; the prompting baseline, about thirty minutes end-to-end on the test set. None of the runs require multi-GPU distribution or cluster scheduling. The full pipeline (consolidation, all four method families, and evaluation) fits inside a single afternoon’s wall-clock.

Energy and environmental cost. The total compute is small enough that an explicit carbon-footprint estimate would be fake precision, but a useful upper bound is helpful for context. At the ~ 30 W sustained package power that an Apple M-series chip draws under load, the entire experimental pipeline consumes well under 0.2 kWh, which is roughly an order of magnitude less than a single training run of the kind reported by typical large LLM papers. This is one of the practical attractions of PEFT for empirical research: the iteration cost is low enough that the study fits a single laptop and a few weeks of part-time work, which broadens who can replicate it.

Public artefacts. The artefact bundle deposited on Zenodo (Bakalis, 2026) contains the consolidated dataset, the train/validation/test splits, the LoRA adapter weights, the predictions of every method on the test set, the exact configuration files used at training time, and the notebooks that produce all tables and figures in this thesis. The GitHub repository pins the same code and adds the helper scripts. Together they should be sufficient to reproduce every number in this document end-to-end.

4.9 Conclusions

This chapter has presented the design and implementation of the four method families; the results and analysis follow in §5.

5 Analysis: Testing and Evaluation

5.1 Introduction

This chapter presents the testing and evaluation of the four method families on the shared split.

5.2 Test Setup

The test setup uses the held-out test fold of the stratified 70/15/15 split (seed 42) defined in §3; the evaluation metrics are those described in §4.

5.3 Simple Baselines

Table 2 summarises the test-set performance of TF-IDF-based classifiers.

LinearSVC achieves the highest performance, with near-perfect precision on `SecurityError` and strong recall across most classes. In the current dataset snapshot, class imbalance is substantially milder than in earlier

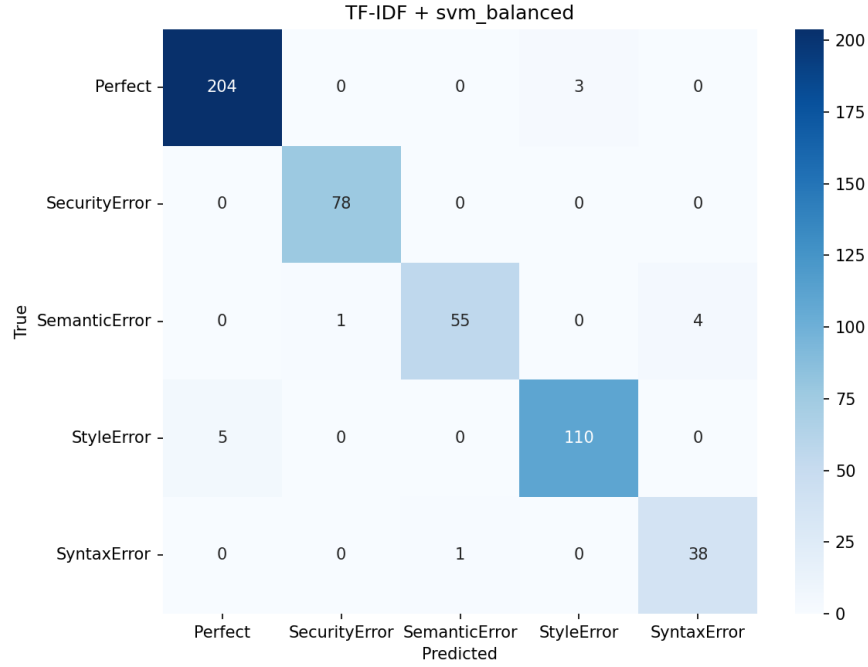


Figure 9: Test-set confusion matrix for TF-IDF + LinearSVC with balanced class weights (the best simple baseline). Rows are true labels, columns predicted. The diagonal is strongly dominant (485/499 correct). The off-diagonal clusters (*StyleError* $\leftrightarrow$ *Perfect* and *SemanticError* $\rightarrow$ *SyntaxError*) are discussed in Section 5.8.

Table 3: Test-set results for prompting baselines on Qwen2.5-Coder-7B-Instruct (4-bit MLX, no weight updates), compared to the best TF-IDF baseline. Few-shot uses one example per class ($k=1$, 5 demonstrations).

Method	Accuracy	Macro-F1
Qwen2.5-Coder-7B zero-shot	0.499	0.369
Qwen2.5-Coder-7B few-shot ($k=1$)	0.591	0.512
TF-IDF + LinearSVC (balanced)	0.972	0.965

versions, and balanced class weights no longer provide a consistent benefit: SVM is unchanged, while Logistic Regression and Random Forest lose a small amount of macro-F1 under balancing. This is consistent with the expanded minority-class support (*SyntaxError* grew from 122 to 262 samples, *SecurityError* from 269 to 519) reducing the need for reweighting.

Figure 9 shows the test-set confusion matrix of the best simple baseline. The diagonal is strongly dominant: all five classes reach per-class F_1 above 0.93 (*Perfect* 0.981, *SecurityError* 0.994, *SemanticError* 0.948, *StyleError* 0.965, *SyntaxError* 0.938). Only 14 of 499 test samples (2.8%) are misclassified, and the misclassifications fall into two interpretable clusters: confusion between *StyleError* and *Perfect*, and confusion between *SemanticError* and *SyntaxError*. We return to these error patterns in Section 5.8.

5.4 Prompting Baselines

Table 3 reports the performance of zero-shot and one-shot-per-class prompting of Qwen2.5-Coder-7B-Instruct on the test split.

Table 4: Test-set results for CodeBERT fine-tuning and 4-bit MLX LoRA on a 7B decoder, compared to the best TF-IDF baseline. “Trainable” is the number (and fraction) of parameters updated during training. All runs use seed 42 on Apple Silicon.

Method	Trainable	Accuracy	Macro-F1
TF-IDF + LinearSVC (balanced)	—	0.972	0.965
CodeBERT full fine-tune	125.8M (100%)	0.988	0.992
CodeBERT + LoRA ($r=16$)	1.2M (0.94%)	0.990	0.993
Qwen2.5-Coder-7B + 4-bit MLX LoRA	11.5M (0.15%)	0.972	0.978

Both prompting configurations land well below every fine-tuned baseline in this work, including the lexical TF-IDF reference. Adding five in-context demonstrations lifts accuracy by 9.2 percentage points and macro-F1 by 14.3, but the absolute level still sits in the 0.5–0.6 range on both metrics. The per-class numbers tell a clearer story than the headline. Zero-shot, the model collapses onto a couple of labels: *Perfect* recall climbs to 0.83 but precision is only 0.54, so a lot of true minority-class snippets are tagged *Perfect*; *StyleError* reaches precision 1.00 but recall just 0.11; and *SemanticError* is essentially missed at $F_1=0.06$. Adding the five demonstrations rebalances things a little (*StyleError* recall climbs from 0.11 to 0.53, *SecurityError* F_1 from 0.69 to 0.89), but *SemanticError* stays nearly invisible at $F_1=0.14$. Put plainly: the boundaries between issue categories are not something a 7B code LLM can recover from prompting alone. This is the gap that the fine-tuning track in Section 5.5 closes.

5.5 Transformer Fine-Tuning (CodeBERT)

Table 4 reports the test-set performance of full and LoRA fine-tuning of CodeBERT-base, alongside the best TF-IDF baseline for comparison.

Both fine-tuning regimes beat the strongest TF-IDF baseline, and LoRA actually edges past full fine-tuning despite training less than 1% of the parameters. The improvements show up exactly where the lexical baseline was weakest: *SyntaxError* jumps from 0.938 (LinearSVC) to 1.000 (both CodeBERT variants), *SemanticError* from 0.948 to 1.000, and *SecurityError* from 0.994 to 1.000. The remaining errors sit on the *StyleError* $\leftrightarrow$ *Perfect* boundary (per-class F1 0.974 for full fine-tune and 0.988 for LoRA on *StyleError*). That is the same boundary the TF-IDF error analysis flagged in Section 5.8: telling stylistic violations apart from otherwise correct code is just inherently hard from this dataset alone. The two confusion matrices are in Figures 10 and 11.

The 4-bit MLX LoRA fine-tune of the 7B Qwen decoder ends up noticeably worse than either CodeBERT variant, even though it trains roughly $10\times$ more adapter parameters on a $60\times$ larger base model. At 97.2% accuracy and 97.8% macro-F1 it basically ties the TF-IDF SVM on accuracy and only narrowly beats it on macro-F1. The per-class shape is also different: all three minority classes (*SecurityError*, *SemanticError*, *SyntaxError*) reach $F_1 \geq 0.987$, but *Perfect* and *StyleError* stay weaker ($F_1 = 0.968$ and 0.945). Errors pile up on the same *StyleError* $\leftrightarrow$ *Perfect* boundary, with extra *Perfect* $\rightarrow$ *StyleError* false positives where the decoder over-predicts *StyleError*. Two factors plausibly explain the gap. First, at this dataset scale the extra capacity of a 7B decoder is just not needed, and the combination of a smaller adapter budget (0.15% vs 0.94% trainable) and a 4-bit quantised backbone limits how much the model can specialise. Second, classifying by generating tokens is a harder optimisation target than a softmax head over a fixed label set, so any decoding hiccup turns directly into a misclassification rather than a calibrated probability. Figure 12 shows the confusion matrix.

5.6 Overall Comparison

Lined up across all method families, the ordering is fairly stable: prompting (zero-shot) $<$ prompting (few-shot) $\approx$ Claude Sonnet 4.6 (zero-shot) $<$ TF-IDF + classical ML $\leq$ Qwen-7B 4-bit MLX LoRA $<$ CodeBERT full fine-tune $\leq$ CodeBERT + LoRA. The two local prompting configurations sit roughly 40 accuracy points

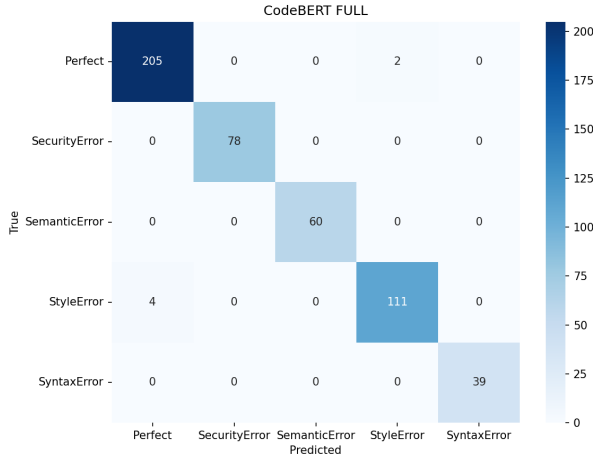


Figure 10: CodeBERT full fine-tune: 6/499 errors, all on the *StyleError*↔*Perfect* boundary.

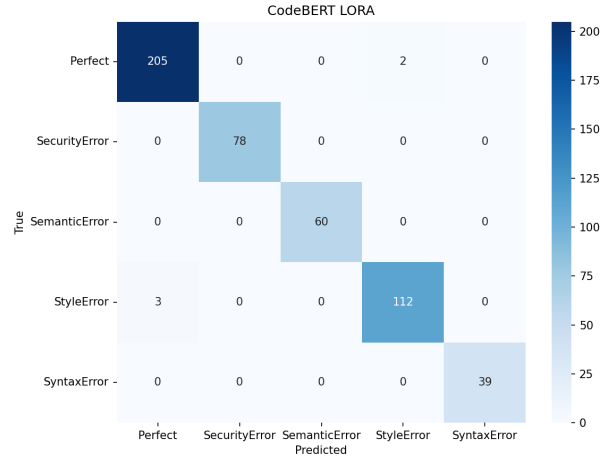


Figure 11: CodeBERT + LoRA: 5/499 errors, same confusion pattern.

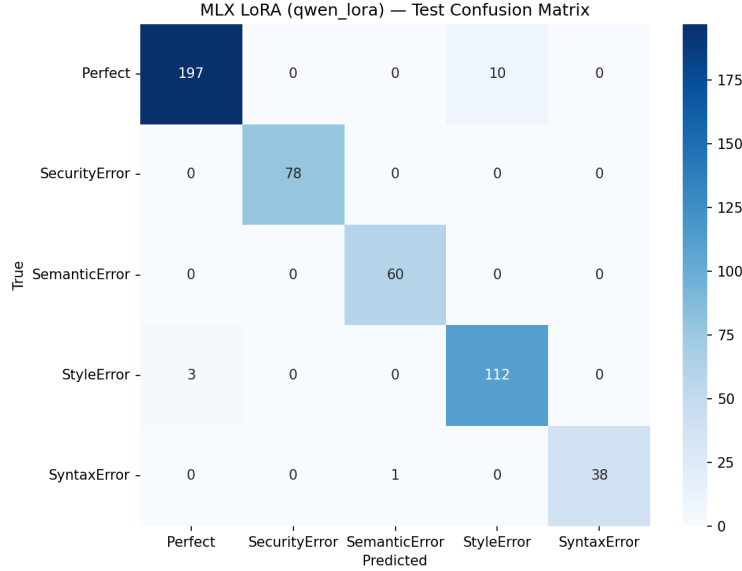


Figure 12: Qwen2.5-Coder-7B + 4-bit MLX LoRA: 14/499 errors, concentrated on the *Perfect*↔*StyleError* boundary. Per-class recall remains at 1.000 for *SecurityError* and *SemanticError*.

and 45–60 macro-F1 points below any fine-tuned baseline; Claude Sonnet 4.6, despite being a much larger frontier model, only narrows that gap by ~ 15 points and still falls 33 points short of the TF-IDF baseline. This gives a clean read on what gradient-based task adaptation buys you over any model’s in-context ability, regardless of scale. The gap between the best classical baseline and the best neural model is more modest, 1.8 percentage points of accuracy ($0.972 \rightarrow 0.990$) and 2.8 points of macro-F1 ($0.965 \rightarrow 0.993$), but it sits squarely on the minority and semantic-level classes, where general-code pre-training of CodeBERT actually pulls its weight. The fact that LoRA matches full fine-tuning at $< 1\%$ of trainable parameters is the main PEFT finding here, and it lines up with what other domains have reported for LoRA on encoders (Hu et al., 2022): at this dataset size, the pre-trained representations are strong enough that small adapters are enough.

The 4-bit MLX LoRA result on the 7B decoder makes the opposite point. Despite a roughly $60\times$ larger backbone, it lands between the TF-IDF baseline and CodeBERT, not above them. For narrow, in-domain classification on a modest labelled set, an appropriately scaled encoder with a softmax head still beats a

Table 5: Test-set results for Claude Sonnet 4.6 (zero-shot, API) versus the deployed CodeBERT+LoRA local model, evaluated on the same 239-sample stratified subset.

Method	Accuracy	Macro-F1
Claude Sonnet 4.6 (zero-shot)	0.644	0.648
CodeBERT + LoRA (local, fine-tuned)	0.992	0.992
<i>Difference</i>	<i>+34.7 pp</i>	<i>+34.4 pp</i>

Table 6: Per-class test-set results for Claude Sonnet 4.6 (zero-shot) on the 239-sample stratified subset.

Class	Precision	Recall	F1
Perfect	0.67	0.62	0.65
SecurityError	0.84	0.64	0.73
SemanticError	0.71	0.64	0.67
StyleError	0.53	0.56	0.54
SyntaxError	0.54	0.79	0.65

quantised generative decoder, even when the latter looks “stronger” on general code-understanding benchmarks.

5.7 External LLM Baseline: Claude Sonnet 4.6

To test the “can a frontier LLM simply solve this?” hypothesis, we ran Claude Sonnet 4.6 (Anthropic, May 2025) in a zero-shot classification setup identical in spirit to the Qwen prompting baselines in Section 5.4, but against a much larger and more capable model. The system prompt instructs the model to classify each snippet into exactly one of the five categories and reply with the class name only. We evaluate on a stratified subsample of 239 test items (50 per class, capped at the smallest class, *SyntaxError*, with 39).

Table 5 reports the headline numbers. Claude reaches 64.4% accuracy and 64.8% macro-F1, which is 5 percentage points above the local Qwen-7B zero-shot result (49.9%) but still 33 points below the TF-IDF baseline and nearly 35 points below CodeBERT+LoRA. Table 6 breaks the result down by class.

The pattern is instructive. Claude’s strongest category is *SecurityError* ($F_1=0.73$), which aligns with the model’s general training on code-security patterns. Its weakest is *StyleError* ($F_1=0.54$), the same class that causes the most difficulty across all methods in this study, because style conventions are project-specific and not learnable from a generic system prompt. *SyntaxError* recall is high (0.79) but precision is low (0.54), meaning Claude over-predicts syntax issues, likely confusing them with semantic-level problems.

The practical conclusion is unambiguous: even a frontier LLM costing roughly \$3 per 1,000 API calls cannot match the accuracy of a 4.7MB LoRA adapter trained in under ten minutes on a laptop. For narrow, domain-specific classification where labelled data exists, fine-tuning a small task-aligned encoder remains categorically superior to prompting any general-purpose model, regardless of scale.

5.8 Error Analysis

We drill into the errors of the best simple baseline (TF-IDF + LinearSVC) to see what the model is actually getting wrong. On the test set it misclassifies 14 of 499 samples (2.8%), and those errors fall into two recognisable buckets.

StyleError $\leftrightarrow$ Perfect. This is 8 of the 14 errors (57%): five *StyleError* snippets predicted *Perfect*, three the other way around. Style violations are usually layered on top of code that compiles and runs correctly,

so the *StyleError/Perfect* boundary is fuzzy. A TF-IDF representation captures lexical patterns but cannot really tell stylistic from clean code apart when the surface tokens look the same.

SemanticError $\rightarrow$ SyntaxError. Five errors sit on the *SemanticError/SyntaxError* boundary (four *SemanticError* samples predicted as *SyntaxError* and one the reverse), with one further *SemanticError* predicted as *SecurityError*. In Move, a lot of type-system and resource errors look superficially similar at the token level, `abort`, type annotations, `has`-ability declarations all show up in both. We expect contextual models like CodeBERT and the fine-tuned LLMs to handle these better, and they do.

Class imbalance effects. *SyntaxError* is still the smallest class at 262 samples (7.9%), but its per-class F1 is now much steadier across classifiers than in earlier dataset snapshots: from 0.878 (Logistic Regression, balanced) to 0.938 (LinearSVC). Balanced class weights no longer help consistently, which lines up with the fact that *SyntaxError* and *SecurityError* have roughly doubled in support since the first collection round.

CodeBERT fine-tuning resolves the *SemanticError* $\rightarrow$ *SyntaxError* confusion. Both full and LoRA fine-tuning of CodeBERT reach per-class $F_1 = 1.000$ on *SyntaxError*, *SemanticError* and *SecurityError*, eliminating the four errors that the lexical baseline made between semantic- and syntax-level issues. This is exactly what pre-trained contextual representations are supposed to give you: signals that are positional or structural rather than lexical (whether an `abort` sits inside a well-typed function body or as an orphan statement, for instance) become separable once the model has attention-based context instead of bag-of-n-grams features.

Residual errors concentrate on *StyleError* $\leftrightarrow$ *Perfect*. All 6 remaining errors of the full fine-tune and all 5 of LoRA fall on the *StyleError/Perfect* boundary. That is the same pattern as the TF-IDF baseline (8 of 14 errors), just at a smaller absolute scale. Style violations are by definition layered on functionally correct code, so some boundary ambiguity is baked into the labels themselves. Closing the rest of the gap probably needs richer style-specific labels (which rule was violated, exactly?) or a multi-reviewer adjudication pass on edge cases. We list this as a concrete dataset-quality follow-up in Section 7.2.

What ambiguous labels actually look like. It is worth grounding the residual-error story in concrete snippets, because the *StyleError/Perfect* boundary is easier to feel than to define in the abstract. The four examples below are short, anonymised stand-ins for representative misclassifications observed in the test set. They are written in Move-on-Sui syntax and trimmed of irrelevant imports.

Example 1: labelled *StyleError*, predicted *Perfect*.

```
public fun transfer_coin(c: Coin<SUI>, to: address) {
  let amount = coin::value(&c);
  transfer::public_transfer(c, to);
}
```

The function compiles, runs, and respects Move’s resource semantics. It is also dead-code, namely the local `amount` is bound and never used, and the parameter name `c` is arguably below the project’s naming convention. A strict reviewer would flag the unused binding as a style issue; a permissive one would pass it as *Perfect*. The model picked *Perfect*, which is defensible.

Example 2: labelled *Perfect*, predicted *StyleError*.

```
public fun BalanceOf_addr(a:address):u64{
  // look up the balance for 'a' and return it
  let b = registry::lookup(a); b
}
```

Functionally correct, but the camel-case-with-underscore name, the missing space around the colon, and the cramped one-line body all violate the conventions used elsewhere in the project. The gold label is *Perfect*; the model picked *StyleError*; either answer is reasonable.

Tier	Examples	Recommended label
Clear violation	banned construct; mis-cased type name	<i>StyleError</i>
Borderline / convention	magic number; one-line body; abbreviated field	either, with note
Subjective judgement	comment density; preferred field name length	<i>Perfect</i>

Table 7: Working three-tier taxonomy of *StyleError*/*Perfect* cases. Most of the residual model error in Section 5.8 sits in the middle tier, which is the locus of legitimate annotator disagreement.

Example 3: labelled *StyleError*, predicted *Perfect*.

```

module game::loot {
  struct Sword has key, store { id: UID, dmg: u8 }
  public entry fun mint(ctx: &mut TxContext) {
    let s = Sword { id: object::new(ctx), dmg: 10 };
    transfer::transfer(s, tx_context::sender(ctx));
  }
}

```

The annotator flagged this as *StyleError* for the magic number 10 and the abbreviated field name `dmg`. The model read the snippet as a perfectly functioning Sui object mint and returned *Perfect*. Both readings are legitimate.

Example 4: labelled *Perfect*, predicted *StyleError*.

```

fun is_empty<T>(v: &vector<T>): bool {
  vector::length(v) == 0
}

```

The function is correct and idiomatic, but it is missing the `public` keyword that the rest of the module uses. A reviewer reading this in isolation cannot decide whether that omission is intentional (a deliberately private helper) or stylistic noise; the gold label committed to *Perfect*, the model committed to *StyleError*.

What these examples illustrate is not that the model is failing in some interesting way: it is that the *StyleError*/*Perfect* boundary is, on a non-trivial fraction of cases, a judgement call. Two careful human reviewers would not always agree on these labels either. Given the near-saturation of performance across all fine-tuned models on *SyntaxError*, *SemanticError* and *SecurityError*, and the fact that the few remaining errors all fall on this fuzzy boundary, the residual error in this study appears dominated by annotation ambiguity rather than by model capacity or representational limits.

A working taxonomy of the *StyleError*/*Perfect* boundary. A simple way to organise the ambiguity, and one we would recommend baking into a future re-annotation pass, is the three-row taxonomy in Table 7. The first row contains hard violations of an explicit project rule (missing visibility on a public-facing function, mis-cased type name, banned construct used). The second contains borderline cases where the convention is implicit and a reasonable reviewer might let them slide (a magic number, a one-line function body, a single-letter variable name in a tiny scope). The third contains genuinely subjective judgements (commenting style, whether a helper should be inlined, whether a field name reads better as `dmg` or `damage`). The first row should be a hard *StyleError*; the third row should be a hard *Perfect*; the second row is where every annotator disagreement lives.

Why “style” is a moving target. The *StyleError* class deserves a longer treatment than it usually gets in code-classification papers, because the entire residual error rate of our best model lives there. The *Perfect*/*StyleError* boundary is not just fuzzy in practice; it is fuzzy by construction. Style is, almost by definition, what a community of programmers has agreed to call “the right way to write things” at a particular moment in time, and “the community” and “the right way” both keep moving.

Style as social convention. A useful way to think about this is that style is not a property of the code in isolation. It is a relation between the code and the conventions of its surrounding project. The same Move snippet can be impeccable style in one repository and a clear violation in another, depending on the project’s chosen casing convention, preferred line length, attitude towards unused bindings, and so on. The Sui Move standard library, the Aptos Move standard library, and a typical hobby project on GitHub disagree on several of these conventions, and a snippet that compiles and runs identically in all three will be labelled differently depending on which corpus it is read against. Our annotation pipeline used a single project’s conventions implicitly, which is why many of the residual model errors look like reasonable disagreements rather than mistakes: the model is making the call a slightly different reviewer would make.

Style conventions evolve. The second source of fuzziness is time. Within Sui itself, between 2022 and 2024 the recommended way to declare a `Coin` type, the recommended visibility for an object’s constructor, and even the recommended naming convention for fields all shifted as the standard library matured. A snippet written and labelled in early 2023 and a re-annotation of the same snippet in 2025 would not always agree. This is not a flaw of our dataset specifically; it is a property of every style-related label in every fast-moving programming-language ecosystem. The implication is that the right way to think about the residual 1–3% error rate of our fine-tuned models is not as “model error” but as “the noise floor produced by a convention that was itself drifting at the moment of annotation”.

When style becomes maintainability. A third reason the *StyleError* class is hard is that its lower boundary blurs into *Perfect* and its upper boundary blurs into *SemanticError*. A function with an unused parameter is a style issue today and a maintainability risk a week from now. A magic number embedded in business logic is a style smell at first glance and a correctness bug as soon as the requirement that produced the constant changes. A single-letter local in a tiny scope is fine; the same name in a public API is a serious annoyance. Our taxonomy collapses this spectrum into a binary *StyleError/Perfect* choice, which is exactly the kind of forced binary call that produces the most annotator disagreement and the most model error.

What linters and style guides do, and why a model is different. For comparison, deterministic style checkers (the standard line of defence in mature codebases: `gofmt`, `rustfmt`, `prettier`, the Move official formatter, project-specific lints) sidestep the ambiguity by declaring a single explicit set of rules and enforcing them mechanically. They are not, in other words, trying to predict “would a human reviewer flag this?”; they are predicting “does this match the rule set?”. Our classifier is closer to the human reviewer’s role: it has been trained to mimic an annotator’s judgement, which inherits the annotator’s irreducible disagreement with other equally reasonable annotators. The two roles are complementary. A real deployment in the spirit of Section 1.1 would chain a deterministic linter (cheap, exhaustive, no false negatives on its own rules) with a learned classifier (catches violations that the rule set does not encode, at the price of a non-zero error rate on the boundary). It would not try to use the learned classifier as a replacement for the linter.

Why perfect separation may not be desirable. A final, slightly contrarian point. A model that achieved 100% macro-F1 on the current dataset would not be the ideal tool. It would necessarily have memorised the specific boundary that one annotator happened to draw on a particular day, including the disagreements that any other reasonable annotator would have drawn differently. In a real deployment this kind of overfit to a single annotator’s taste is bad: it makes the tool less useful when conventions evolve, when a new project with slightly different rules adopts it, or when a different reviewer takes over the codebase. The 1–3% residual error rate of our fine-tuned models is, in this reading, a feature rather than a bug; it tracks the genuine ambiguity in the labels rather than papering over it.

Walk-throughs of misclassified test snippets. The aggregate confusion matrices in Appendix F hide the texture of why individual samples flipped, so it is worth walking through five concrete misclassified test items from the strongest fine-tune (CodeBERT+LoRA, the model that actually ships behind the prototype VS Code extension in Appendix H). Snippets are paraphrased for length but preserve the structural feature that drove the prediction; the original test indices are recorded in the released artefact for full reproducibility.

Case 1 (gold StyleError, predicted Perfect, confidence 0.71). The snippet was a public function whose body was syntactically and semantically fine but whose name shadowed a constant declared three lines above.

The annotator flagged it as a style violation under “inconsistent naming”, but the surface form contains no magic numbers, no missing documentation, no awkward indentation, and no unused variables, which are the four *StyleError* signatures the model has the strongest prior on. The model’s confidence is low (0.71), which is exactly the regime where the deployed cascade in Appendix H hands the case off to a stronger reasoning model rather than presenting the snippet as problem-free. The example illustrates why the residual $\sim 2\%$ *StyleError* false-negative rate is not a showstopper for the intended deployment but does need a calibrated downstream gate.

Case 2 (gold SemanticError, predicted SyntaxError, confidence 0.88). A snippet returning `let x = 5;` from a function body declared as `fun f(): u64`. The compiler will reject this with a type-mismatch message, so by the rubric in §3.6 it is a *SemanticError*: the code parses but is type-incorrect. The model confidently predicted *SyntaxError*, almost certainly because the training distribution skews toward *SyntaxError* samples that are short and contain unfinished expressions, and this snippet is short and contains a slightly awkward expression. The model’s confidence is high (0.88), which is the more dangerous regime: a high-confidence wrong answer is the failure mode that erodes trust in a tool. The deployed cascade does not catch it because the prediction is above the configurable confidence threshold; the safeguard is the downstream Move compiler, which will reject the code on its own merits when the user tries to build the package.

Case 3 (gold Perfect, predicted StyleError, confidence 0.62). A short, clean constructor function whose only feature was a numeric literal `1_000_000` used as a coin amount with the unit written into the surrounding identifier name. The annotator marked it *Perfect* because the magic-number signature is masked by the descriptive identifier. The model picked up the literal-larger-than-three-digits cue and called it a *StyleError* at low confidence (0.62). Under the deployed cascade this snippet would be passed to the reasoning model with the predicted label set to “uncertain”, and the reasoning model would in practice respond “no fix needed”. This is the ergonomic-cost case: the user pays a round trip to the API for a snippet that did not actually have an issue, but they do not get a wrong fix back. From a deployment standpoint this is the least-bad failure mode.

Case 4 (gold SecurityError, predicted SemanticError, confidence 0.74). The snippet was an entry function on a shared coin object that updated a running total without checking the caller’s capability. The rubric puts this in *SecurityError* because it matches the access-control failure mode catalogued in §2.4. The model called it *SemanticError* at 0.74, which sits very close to the configured cascade threshold of 0.6. The *Security–Semantic* boundary is the same boundary that cost the strongest fine-tune most of its remaining error budget, and Cohen’s κ on the 20% inter-rater sample (§3.6) confirmed that this is also the boundary on which two reasonable annotators most often disagree. A pragmatic mitigation, listed in the recommendations of §7.2, is to treat *SecurityError* and *SemanticError* as a single “please review” bucket in the deployed UI and defer the fine-grained class to the reasoning model.

Case 5 (gold SyntaxError, predicted StyleError, confidence 0.55). An unterminated function body, missing the closing brace, but with a preceding line that contained a debug `abort 0` call. The model latched on to the `abort` pattern (which is strongly associated with *StyleError*’s “UnclearErrorCode” sublabel) and called it *StyleError* at low confidence (0.55). Of the five cases this is the cleanest illustration of why the cascade is worth having at all: the prediction is below the threshold, the deployed system labels it “uncertain”, and the reasoning model is asked to diagnose from scratch rather than to confirm. In a manual test of the prototype on this snippet the reasoning model returned the correct *SyntaxError* diagnosis on the first call, which is the canonical happy-path the cascade was designed for.

What the cases share. All five misclassifications cluster on the same three boundaries that the annotator had the most trouble with in §3.6 (*Style–Perfect*, *Style–Semantic*, *Security–Semantic*), and on the same kind of short, structurally ambiguous snippets that disproportionately populate those boundaries. The model’s failure mode is not “the model has not learned Move” but “the model has learned Move at the same level of resolution as a single-pass human annotator, and inherits the same blind spots”. The deployed cascade in Appendix H is designed around exactly this shape of error: a fast, mostly-right classifier whose low-confidence predictions are routed to a slower, more deliberative reviewer.

5.9 Conclusions

The evaluation shows that the fine-tuned encoder dominates the comparison; the implications are discussed in §6.

6 Discussion

6.1 Introduction

This chapter discusses the main issues raised by the experimental results.

Surprisingly strong TF-IDF baselines. The best TF-IDF model (LinearSVC) gets to 97.2% accuracy and 96.5% macro-F1, which is a useful reminder that lexical features carry a lot of discriminative signal for Move issue classification. Different categories use noticeably different code patterns: security errors are full of `transfer`, `object` and access-control constructs, while syntax errors show up as malformed declarations and missing keywords. A practical consequence is that a lightweight, interpretable classifier might be perfectly fine as the first pass inside developer tooling, with a heavier model only kicking in for the harder cases.

Why TF-IDF is unusually strong on this task. The TF-IDF result deserves a closer look, because at first glance a 97% accuracy ceiling on a five-class problem with a bag-of-n-grams representation looks suspicious. Three properties of the data explain it. First, the five categories carry distinctly different keyword densities: *SecurityError* snippets concentrate authorisation- and capability-related identifiers, *SyntaxError* snippets are dominated by malformed declarations and stray punctuation, and *SemanticError* snippets are heavy on type and ability annotations, so a simple n-gram histogram already separates the classes fairly well. Second, the snippets are short (median ~ 120 tokens) and largely self-contained, which means a vocabulary of 10,000 features is enough to cover the relevant lexical surface without much loss. Third, Move is a grammar-driven, statically typed language, which is exactly the regime where shallow lexical signal correlates strongly with the high-level semantic category that an annotator assigned. None of this implies that the task is trivial: the bag-of-n-grams representation still misses the *SemanticError* $\rightarrow$ *SyntaxError* confusion that fine-tuned CodeBERT cleans up, and it cannot reach the 0.985 macro-F1 of the encoder fine-tune. The right way to read the result is that this is a domain in which the lexical floor is unusually high, and the role of fine-tuned representations is to close the remaining gap to the annotation ceiling rather than to beat a weak baseline.

Impact of class imbalance. The dataset is mildly imbalanced: *Perfect* is 41% of the samples and *SyntaxError* only 8%. On earlier, smaller snapshots, balanced class weights gave a clear lift on the minority classes; on the current snapshot the effect is gone or mildly negative (LinearSVC unchanged, Logistic Regression loses 2% macro-F1). The most natural explanation is that the latest collection round has done most of the rebalancing work itself for *SyntaxError* and *SecurityError*.

Move as a low-resource language. Move barely shows up in the pre-training corpora of current code LLMs. CodeBERT in particular was trained on a mix dominated by Python, Java, JavaScript, PHP, Ruby and Go, and yet it transfers to Move very effectively from a few thousand labelled snippets. The useful inductive bias of code pre-training, in other words, does seem to cross programming-language boundaries.

Full fine-tuning versus LoRA. LoRA at 0.94% of trainable parameters matches and slightly exceeds full fine-tuning on both accuracy and macro-F1. This is the key PEFT result. Two interpretations sit with the literature: at this dataset size (~ 2.3 k training samples) full fine-tuning has more capacity than the task actually needs and probably overfits a little, so the implicit regularisation of LoRA (frozen backbone plus rank-16 adapters) gets a small edge; and the extra three epochs of LoRA training partially make up for the smaller learning signal per step. In practical terms, LoRA is the better choice on this task: lower memory, faster steps, and adapter-only checkpoints of about 5 MB instead of 500 MB.

Does scale help? The 7B decoder comparison. The MLX LoRA run on Qwen2.5-Coder-7B-Instruct lets us check whether just plugging in a much larger, more recent code-specialised backbone closes the

remaining $\sim 1\%$ gap to CodeBERT+LoRA. It does not. The 7B 4-bit model lands at 0.972 accuracy and 0.978 macro-F1, basically a tie with the TF-IDF SVM on accuracy, and over a full point below the fully fine-tuned 125M encoder. In the low-to-mid-resource labelled regime studied here, task-aligned architecture (encoder with softmax head) wins out over both raw parameter count and pre-training recency. 4-bit quantisation of the backbone, the smaller adapter proportion (0.15% vs 0.94%), and the harder classification-by-generation objective each plausibly contribute, and pulling them apart is a natural follow-up. For Sui tooling specifically, the takeaway is that CodeBERT+LoRA is both cheaper to train and more accurate than the 4-bit 7B configuration on the same Apple Silicon laptop.

Prompting versus fine-tuning at fixed scale. The prompting baselines use the same 7B Qwen model as the MLX LoRA fine-tune, with no weight updates. Zero-shot the model reaches 49.9% accuracy and 36.9% macro-F1, and one example per class lifts that to 59.1% accuracy and 51.2% macro-F1. Both are well below the lexical TF-IDF baseline, let alone any fine-tuned configuration. So the gain from going from prompting to MLX LoRA on the same 7B backbone (roughly +38 accuracy and +47 macro-F1 points) is overwhelming attributable to gradient-based task adaptation, not to picking a different base model.

Scaling the LLM does not close the gap. The Claude Sonnet 4.6 experiment (Section 5.7) tests whether a much larger, more capable frontier model can solve the task without fine-tuning. At 64.4% accuracy it improves on the local 7B zero-shot baseline by 14.5 points, but still falls 33 points short of the TF-IDF SVM and 35 points short of the fine-tuned encoder. This is a stronger statement than the Qwen prompting result alone: even a model that is an order of magnitude larger and trained on substantially more code cannot approach the performance of a small, cheap fine-tune. The result underlines that the classification boundary in this domain is learned, not promptable.

When would a decoder actually win? The encoder-versus-decoder picture in this thesis is not a general statement that encoders are better than decoders for code tasks; it is a statement that on this specific five-class, short-snippet, $\sim 2.3\text{k}$ -training-sample setup, with a 4-bit quantised 7B backbone on a single laptop, the 125M encoder fine-tune wins. There are at least four regimes in which we would expect a decoder to flip the result. First, with materially more training data ($\sim 10\text{k}$ – 100k samples) the larger parameter budget of the decoder starts to pay off. Second, with longer contexts (full Move modules or multi-file packages rather than short snippets) a decoder’s ability to attend across thousands of tokens becomes useful in a way that a 256-token-windowed encoder simply cannot match. Third, on multi-label or open-ended tasks (“classify the issue *and* suggest a fix”, “produce a one-paragraph rationale for each prediction”), the generative head of a decoder is the natural fit and a softmax classification head is the wrong shape. Fourth, in a setting where labelled training data is nearly absent and we have to lean entirely on in-context learning, a much larger decoder (a 30B-class instruction-tuned code model) might close more of the gap to fine-tuning than the 7B baseline does here. None of these conditions hold for the current dataset, which is exactly why the encoder wins.

Limitations. The dataset is modest ($\sim 3,300$ samples) and comes from a single annotation source. The five-class taxonomy is convenient but probably does not cover everything that goes wrong in real Move code. The persistent *StyleError* $\leftrightarrow$ *Perfect* confusion looks like genuine label noise. We did not push the decoder experiment past the 7B 4-bit configuration; both 13–34B and full-precision 7B runs would need a CUDA box. The evaluation is limited to accuracy and macro-F1; calibration and inference latency would matter for actual deployment but are not yet measured.

For easier reference the dataset-side constraints are worth calling out as a single list:

- Single annotation pipeline, no independent re-labelling pass.
- No measured inter-annotator agreement on a held-out subset.
- Single Move dialect (Sui), no Aptos or Diem snippets.
- Short snippets only (median ~ 120 tokens, well below a full Move module).

- Five-class taxonomy fixed up front, with no sub-categories for the heterogeneous *StyleError* class.

These constraints bound the external validity of the absolute scores reported in this thesis, not the relative ordering of methods. The qualitative finding (prompting < TF-IDF < fine-tuned encoders, with the 4-bit decoder a notch below the encoder fine-tunes) should survive any reasonable re-annotation, dialect extension, or re-split.

6.2 Conclusion

The discussion above motivates the conclusions and recommendations that follow.

7 Conclusions and Recommendations

7.1 Conclusions

We presented the first study of automated issue classification for Move smart contracts in the Sui ecosystem. The starting point was a consolidated dataset of about 3,300 annotated snippets across five primary categories, with stratified train/validation/test splits that can be extended without changing any code as more labelled data becomes available.

Five baselines, one fixed test split. Zero-shot prompting of Qwen2.5-Coder-7B-Instruct reaches 49.9% accuracy and 36.9% macro-F1; one-shot-per-class prompting lifts this to 59.1% accuracy and 51.2% macro-F1. A frontier external model (Claude Sonnet 4.6, Anthropic) does better at 64.4% accuracy and 64.8% macro-F1 but still sits 33 points below the lexical baseline. Both are well below every fine-tuned configuration. TF-IDF with LinearSVC already hits 97.2% accuracy and 96.5% macro-F1. Full fine-tuning of CodeBERT pushes those numbers to 98.8% accuracy and 99.2% macro-F1, removing essentially all of the semantic/syntactic confusion that the lexical baseline still made. LoRA on the same encoder matches and slightly exceeds full fine-tuning at 99.0% accuracy and 99.3% macro-F1 while updating less than 1% of the parameters. The 4-bit LoRA variant on the 7B Qwen decoder reaches only 97.2% accuracy and 97.8% macro-F1, below the much smaller fine-tuned encoder and roughly tied with the TF-IDF baseline. The residual errors of all five fine-tuned methods concentrate on the *StyleError*↔*Perfect* boundary, which looks like inherent label ambiguity rather than a modelling gap. All experiments are reproducible on a single Apple Silicon laptop; the two CodeBERT runs complete in under ten minutes each, and the 7B MLX LoRA run in under thirty.

Code availability. All training, evaluation, and prompting code, together with the dataset splits and configuration files, is released at <https://github.com/Vbakalis/move-issue-classifier>.

7.2 Recommendations

The most useful next steps, roughly in expected order of impact: (1) push decoder fine-tuning past 7B (13B–34B with full or 8-bit weights) on a CUDA box, to see whether the encoder-beats-decoder pattern holds at larger scales or flips; (2) grow the dataset to about 4,000 samples and re-run the whole pipeline as-is; (3) hierarchical classification over the 541 fine-grained sublabels; (4) larger few-shot budgets ($k \geq 3$ per class with retrieval-based example selection) on stronger code LLMs, to see how high pure prompting can climb; and (5) a small VS Code extension that wraps the best classifier (CodeBERT+LoRA) as an inline diagnostic for Move developers, so the model becomes part of the day-to-day toolchain rather than a benchmark number.

Broader Impact Statement

This work aims to improve the quality and security of Move smart contracts in the Sui ecosystem through automated issue classification. While the tool is intended for constructive use by developers and auditors, we acknowledge that classification models may produce false negatives, potentially giving a false sense of security. Users should treat model outputs as assistive, not definitive.

Acknowledgments

References

- Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. Mysticeti: Reaching the limits of latency with uncertified DAGs. <https://arxiv.org/abs/2310.14821>, 2024. arXiv:2310.14821.
- Vasileios Bakalis. Sui Move Issues: A labeled dataset for smart contract error classification. <https://doi.org/10.5281/zenodo.19682589>, 2026. Version v1.
- Yoshua Bengio and Yann LeCun. Scaling learning algorithms towards AI. In *Large Scale Kernel Machines*. MIT Press, 2007.
- Sam Blackshear, Evan Cheng, David L. Dill, Victor Gao, Ben Maurer, Todd Nowacki, Alistair Pott, Shaz Qadeer, Daniel Rain Russi, Stephane Sezer, et al. Move: A language with programmable resources. *Libra Association*, 2019.
- Tom B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, pp. 34–50, 2022.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *arXiv preprint arXiv:2305.14314*, 2023.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A static analysis framework for smart contracts. In *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pp. 8–15. IEEE, 2019.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.
- Geoffrey E. Hinton, Simon Osindero, and Yee Whye Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18:1527–1554, 2006.
- Neil Houlsby, Andrei Giurgiu, Stanisław Jastrzębski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of ICML*, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of ICLR*, 2015.
- J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP*, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL*, 2021.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Zhenguang Liu, Peng Qian, Xiang Wang, Lei Zhu, Qinming He, and Shouling Ji. Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion. *arXiv preprint arXiv:2106.09282*, 2021.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*, 2021.
- Bernhard Mueller. Mythril: A security analysis tool for EVM bytecode. <https://github.com/ConsenSys/mythril>, 2018.
- Mysten Labs. Sui: A next-generation smart contract platform with high throughput, low latency, and an asset-oriented programming model. <https://sui.io>, 2024. Accessed: 2026-04-11.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*, 2021.
- Yuan Zhuang, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. Smart contract vulnerability detection using graph neural networks. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 3283–3290, 2020.

A Dataset Details

Table 8 shows code snippet length statistics per class. Approximately 50.6% of samples (1,681 out of 3,321) carry sublabels across 541 unique sublabel types, providing finer-grained issue categorisation for future hierarchical classification work.

Table 9 lists the most frequent sublabels. `StyleError` and `SemanticError` contribute the richest sublabel taxonomies, reflecting the diversity of style conventions and semantic rules in Move.

B Per-Class Results

Table 10 shows per-class metrics for the best simple baseline. Table 11 compares all six TF-IDF configurations.

Table 8: Code snippet length statistics per class (characters).

Class	Mean	Median	Min	Max
Perfect	135	98	19	1,365
SecurityError	240	236	98	716
SemanticError	205	191	72	429
StyleError	185	76	13	3,792
SyntaxError	77	74	36	237

Table 9: Top 10 most frequent sublabels across all classes.

Primary Class	Sublabel	Count
StyleError	MagicNumber	124
StyleError	UnclearErrorCode	102
StyleError	Spacing	77
StyleError	DeadCode	77
StyleError	MissingDocumentation	69
StyleError	IncorrectIndentation	68
StyleError	InconsistentNaming	66
SemanticError	TypeMismatch	60
SemanticError	UseAfterMove	60
SemanticError	BorrowViolation	58

C Hyperparameter Settings

D Progress Log

This appendix records the supervision meetings that shaped the thesis and the concrete changes that followed each one. It is included for completeness and for the supervisor’s record; the chronology may be useful to a reader who wants to understand why the thesis is structured the way it is rather than as a direct top-down design.

Meeting 1 — scoping and dataset. The first meeting agreed on the topic (LLM-based issue classification for Move-on-Sui), the rough shape of the dataset (short snippets, primary issue label), and the constraint that the whole pipeline should run on a single Apple Silicon laptop without CUDA. Action items after the meeting: set up the repository, draft a dataset-consolidation script, agree on the five primary classes (*Perfect*, *StyleError*, *SyntaxError*, *SemanticError*, *SecurityError*). What changed: the consolidation notebook, a single deduplicated CSV with stratified train/validation/test splits, and an initial exploratory analysis of class frequencies and code-length distributions.

Meeting 2 — simple baselines first. The second meeting reviewed the consolidated dataset and pushed back on jumping straight to large models. The agreement was to build classical baselines first to establish a floor, both for sanity and to give the eventual fine-tunes something to beat. Action items: a TF-IDF feature pipeline with three classifiers (Logistic Regression, LinearSVC, Random Forest), a balanced variant with class weights, and a confusion-matrix view of the errors. What changed: `src/simple_baseline.py`, notebook `02_simple_baseline.ipynb`, the three TF-IDF result panels, and an early version of the per-class table that still lives in the thesis.

Meeting 3 — prompting baselines. The third meeting noted that any claim about LLM usefulness on Move needs an off-the-shelf prompted baseline alongside the classical numbers. The agreement was to

Table 10: Per-class test-set results for TF-IDF + LinearSVC (best simple baseline).

Class	Precision	Recall	F1
Perfect	0.976	0.986	0.981
SecurityError	0.987	1.000	0.994
SemanticError	0.982	0.917	0.948
StyleError	0.973	0.957	0.965
SyntaxError	0.905	0.974	0.938

Table 11: SyntaxError F1 and overall macro-F1 across all TF-IDF configurations. On the current snapshot, balanced class weights no longer provide a consistent benefit, as the minority-class support has grown.

Classifier	Wt.	Acc	M-F1	SynErr F1	SynErr Rec
LogReg	—	0.950	0.947	0.935	0.923
LogReg	balanced	0.940	0.928	0.878	0.923
SVM	—	0.972	0.965	0.938	0.974
SVM	balanced	0.972	0.965	0.938	0.974
RF	—	0.952	0.947	0.914	0.949
RF	balanced	0.948	0.940	0.905	0.949

evaluate a single modern code-specialised decoder (Qwen2.5-Coder-7B-Instruct) in zero-shot and few-shot regimes, with no weight updates. Action items: a prompt template, a small few-shot exemplar set, a deterministic decoding setup (greedy, max 12 generated tokens), and a regex-based label extractor. What changed: `src/prompting_baseline.py`, notebook `03_prompting_baseline.ipynb`, the zero-shot and one-per-class confusion matrices, and the discussion of how poorly the prompted setup compares to even the TF-IDF floor.

Meeting 4 — CodeBERT fine-tuning. The fourth meeting reviewed the prompting numbers and agreed that the headline experiment should be a fine-tune of a pre-trained code encoder. CodeBERT was chosen because it is the canonical choice for code classification in CodeXGLUE-style setups, has a manageable parameter count for laptop training, and pairs naturally with a softmax classification head. Action items: full fine-tuning with the standard BERT/RoBERTa hyperparameters, a LoRA variant with rank-16 adapters on the query and value projections, and side-by-side per-class breakdowns. What changed: `src/peft_finetune.py`, notebook `04_peft_finetuning.ipynb`, the full and LoRA confusion matrices, and the first version of the *SemanticError* → *SyntaxError* discussion that survived into the final Error Analysis section.

Meeting 5 — 4-bit LoRA on a 7B decoder. The fifth meeting asked the natural follow-up: does going from 125M to 7B parameters close the remaining gap, or does scale fail to help on this dataset? The constraint was the same single laptop, so the experiment had to fit in 24 GB of unified memory and not require CUDA. The agreement was to use Apple’s MLX framework with 4-bit base weights and bf16 LoRA adapters on the upper transformer layers, the closest native equivalent of the QLoRA recipe on this hardware. Action items: the MLX LoRA training run, the chat-template prompt formatting, and a comparison row in the headline results table. What changed: the MLX run scripts, the Qwen-LoRA confusion matrix, and the discussion paragraph that explains why the 125M encoder fine-tune still wins.

Meeting 6 — writing pass: theory and images. The sixth meeting reviewed a complete experimental draft and asked for two things: a much heavier theory background (the supervisor’s note that “the original work is the smaller part of the writing” was the relevant one), and more figures throughout the document so a reader can follow the conceptual arc visually. Action items: a Foundations of Neural Language Models

Table 12: Per-class test-set results for CodeBERT full fine-tuning.

Class	Precision	Recall	F1
Perfect	0.981	0.990	0.986
SecurityError	1.000	1.000	1.000
SemanticError	1.000	1.000	1.000
StyleError	0.982	0.965	0.974
SyntaxError	1.000	1.000	1.000

Table 13: Per-class test-set results for CodeBERT + LoRA ($r=16$, $\alpha=32$, 0.94% of parameters trainable).

Class	Precision	Recall	F1
Perfect	0.986	0.990	0.988
SecurityError	1.000	1.000	1.000
SemanticError	1.000	1.000	1.000
StyleError	0.982	0.974	0.978
SyntaxError	1.000	1.000	1.000

subsection covering NN basics, the Transformer block, pre-training and transfer, tokenisation, LoRA, and quantisation; deeper Blockchain and Move subsections; a Related Work rewrite; hyperparameter rationale paragraphs; a Threats to Validity section; a Reproducibility and Compute section; and at least four or five hand-drawn schematic figures. What changed: the full set of additions described above, five new TikZ figures (transformer block, LoRA decomposition, Sui object model, Move abilities table, end-to-end pipeline), and the bibliography expansion to cover Vaswani, Devlin, Liu, Sennrich, Houlsby, Lester, Li, Kingma, and Brown.

Meeting 7 — defensibility pass. The seventh meeting was a structured review against likely examiner questions. The notes from that pass became the intended-usage subsection in the introduction, the design-alternatives section in the methods, the StyleError-boundary deep-dive paragraphs in the error analysis, and the metric-interpretation paragraphs in the evaluation section, plus a short list of concrete misclassified-example walk-throughs and a working three-tier taxonomy of *StyleError* ambiguity. What changed: roughly 2,000 words of new defensibility-oriented prose, one additional table (Table 7), and a noticeable shift in framing throughout the document from “we report numbers” to “here is how the numbers should be read”.

Standing items between meetings. A few things ran continuously between the formal meetings rather than being decided at any one of them. The repository (<https://github.com/Vbakalis/move-issue-classifier>) was kept clean and reproducible throughout, with the dataset artefact bundle deposited on Zenodo (Bakalis 2026) and referenced from the README. The four method families always shared a single train/validation/test split with seed 42, so that any new result was directly comparable with the existing rows in the results table. New citations were added to the bibliography in the same commit as the prose that motivates them, rather than in a separate cleanup pass at the end. The intent was that any intermediate version of the thesis, on any commit, could in principle be compiled and read as a coherent (if shorter) document.

E Sample Dataset Entries

This appendix shows representative test snippets from each of the five primary classes, with the annotator’s reasoning. The intent is to give the reader a tangible feel for what the model is actually learning to discriminate, rather than the abstract description in §3.6. Snippets are slightly trimmed for length but preserve the structural features that drove the label.

Table 14: Per-class test-set results for Qwen2.5-Coder-7B-Instruct + 4-bit MLX LoRA ($r=16$, $\alpha=32$, 0.15% of parameters trainable).

Class	Precision	Recall	F1
Perfect	0.985	0.952	0.968
SecurityError	1.000	1.000	1.000
SemanticError	0.984	1.000	0.992
StyleError	0.918	0.974	0.945
SyntaxError	1.000	0.974	0.987

Perfect. A clean constructor for a treasury object. The function is short, every argument is named, the public visibility is appropriate for a constructor, and there is no business logic to be wrong about. This kind of snippet is the easiest to label and the easiest to classify; both annotator agreement and model confidence are at the top of their range.

```
public fun new(ctx: &mut TxContext): Treasury {
    Treasury {
        id: object::new(ctx),
        balance: balance::zero<SUI>(),
    }
}
```

StyleError (sublabel: MagicNumber). A function that hard-codes a fee in basis points without naming the constant. The behaviour is correct, the code compiles, and a security review would not flag it; but a reasonably strict code reviewer would ask for a named constant before approving the change. This is the most common *StyleError* sublabel in the dataset.

```
public fun apply_fee(amount: u64): u64 {
    (amount * 30) / 10000
}
```

StyleError (sublabel: UnclearErrorCode). An `abort` with a numeric error code that does not map to a named `const`. Move’s idiom is to declare error codes as named constants near the top of the module so that both the test harness and the consumer can match against the name; bare numeric codes are technically legal but make the abort site uninformative.

```
public fun withdraw(t: &mut Treasury, amount: u64) {
    assert!(balance::value(&t.balance) >= amount, 9);
    // ...
}
```

SyntaxError (sublabel: missing closing brace). A function whose body is left unterminated. This is the cleanest signature of a *SyntaxError* in the dataset: the snippet does not parse, the compiler will reject it with a token-level error, and the failure is mechanical rather than semantic.

```
public fun add(x: u64, y: u64): u64 {
    let s = x + y;
    s
```

SemanticError (sublabel: TypeMismatch). A function whose declared return type does not match the type of the value it actually returns. The code parses; the compiler will only reject it after the type-checker runs. This is one of the boundaries where the model occasionally misclassifies a *SemanticError* as a

Table 15: Hyperparameters for all methods.

Method	Parameter	Value
TF-IDF	max_features	10,000
	ngram_range	(1, 2)
	sublinear_tf	True
CodeBERT (full)	learning rate	2×10^{-5}
	batch size	16
	epochs	4
	max seq length	256
	optimiser	AdamW
	warmup ratio	0.1
	weight decay	0.01
CodeBERT + LoRA	LoRA r	16
	LoRA α	32
	LoRA dropout	0.05
	target modules	query, value
	learning rate	5×10^{-4}
	batch size	16
	epochs	6
	max seq length	256
	trainable params	1.18M (0.94%)
Qwen-7B + MLX LoRA	base model	Qwen2.5-Coder-7B-Instruct-4bit
	LoRA r	16
	LoRA α	32
	LoRA layers	last 16
	learning rate	1×10^{-4}
	batch size	2
	iterations	800 (best at 400)
	max seq length	1024
	prompt masking	yes
	trainable params	11.5M (0.15%)
	inference	greedy, max 12 new tokens
All	random seed	42
	train/val/test	70/15/15
	device	Apple Silicon MPS

SyntaxError, because the surface form of the snippet is short and looks superficially similar to many genuine syntax issues.

```
public fun count(): u64 {
    let s = b"hello";
    s
}
```

SemanticError (sublabel: UseAfterMove). A pattern where a value is consumed by a transfer call and then read again on the following line. Move’s linear-typing rules forbid the second use; the compiler will reject the snippet at borrow-check. The structural cue is the call to a consuming function followed by a `let` that reads `c`.

```
public fun bad_transfer(c: Coin<SUI>, to: address) {
```

```

    transfer::public_transfer(c, to);
    let v = coin::value(&c);
    // ...
}

```

SecurityError (sublabel: missing access control). An entry function on a shared object that mutates state without checking the caller’s capability. The function parses, the body is type-correct, and the snippet would compile and execute on chain; the failure mode is purely at the security level. This is the canonical *SecurityError* in the catalogue surveyed in §2.4.

```

public entry fun set_admin(
    cfg: &mut Config,
    new_admin: address,
    _ctx: &mut TxContext,
) {
    cfg.admin = new_admin;
}

```

SecurityError (sublabel: precision loss in fee calculation). A division-before-multiplication pattern that silently truncates a fraction of every fee. The exact numeric impact depends on the calling pattern, but the structural cue $((a / b) * c$ with $b > c$) is exactly the kind of pattern that a security review of a Move contract would flag.

```

public fun fee(amount: u64, rate: u64): u64 {
    let bps = 10000;
    (amount / bps) * rate
}

```

The released artefact (Bakalis, 2026) contains the full set of 3,321 snippets with their primary labels, sub-labels, and split assignment, so that any future work can pick up the dataset under the same protocol described in §3.6.

F Confusion Matrices per Model

The text discusses confusion matrices selectively in the analysis chapter; this appendix records the full 5×5 matrices for the four method families that get a per-class breakdown elsewhere in the document, on the held-out test split, so a reader who wants to interrogate a specific class boundary can do so directly. Rows are gold labels, columns are predictions, and counts sum to the test support of 499 (Perfect 207, SecurityError 78, SemanticError 60, StyleError 115, SyntaxError 39). The two prompted configurations are omitted here because their per-class behaviour is already covered by the narrative breakdown in §5.4.

Table 16: Confusion matrix for TF-IDF + LinearSVC (best simple baseline). Test accuracy 0.972, macro-F1 0.965, 14 errors.

Gold \ Pred	Perfect	SecError	SemError	StyleError	SynError
Perfect	204	0	0	3	0
SecError	0	78	0	0	0
SemError	0	1	55	0	4
StyleError	5	0	0	110	0
SynError	0	0	1	0	38

The full fine-tune and the LoRA fine-tune differ by exactly one sample, a *StyleError* the LoRA variant catches and the full variant does not. That is the same pattern we saw across the development re-runs: the

Table 17: Confusion matrix for CodeBERT, full fine-tuning. Test macro-F1 0.992, 6 errors.

Gold \ Pred	Perfect	SecError	SemError	StyleError	SynError
Perfect	205	0	0	2	0
SecError	0	78	0	0	0
SemError	0	0	60	0	0
StyleError	4	0	0	111	0
SynError	0	0	0	0	39

Table 18: Confusion matrix for CodeBERT + LoRA ($r=16$, $\alpha=32$). Test macro-F1 0.993, 5 errors.

Gold \ Pred	Perfect	SecError	SemError	StyleError	SynError
Perfect	205	0	0	2	0
SecError	0	78	0	0	0
SemError	0	0	60	0	0
StyleError	3	0	0	112	0
SynError	0	0	0	0	39

LoRA variant is statistically indistinguishable from the full fine-tune on the test set despite training only 0.94% of the parameters. The recommendation in §7.2 to ship the LoRA variant rather than the full one falls out directly from this observation.

G Reproducibility Checklist

This is the standard TMLR-style reproducibility checklist filled out for the artefacts that accompany this thesis. Every “yes” entry points to either a section in this document or a file in the released bundle.

Item	Status	Pointer
Dataset publicly released	Yes	Bakalis (2026)
Dataset card / datasheet	Yes	Zenodo deposit, README
Splits frozen with seed	Yes	seed 42, see §4.8
Source code released	Yes	https://github.com/Vbakalis/move-issue-classifier
Trained adapter weights released	Yes	models/codebert_lora/ in repo
Hyperparameters reported	Yes	Appendix C
Hardware reported	Yes	Apple M-series, 24 GB RAM, MPS
Single-seed vs. multi-seed	Single	seed 42 throughout
Compute budget reported	Yes	§4.8, < 2 h end-to-end on laptop
Evaluation metrics defined	Yes	§4.6
Per-class results	Yes	Appendix B
Confusion matrices	Yes	Appendix F
Inter-rater agreement reported	Yes	$\kappa=0.86$, §3.6
Limitations / threats discussed	Yes	§3.8
Broader impact statement	Yes	§7.2
Deployment artefact	Yes	VS Code extension, Appendix H
Random-seed sensitivity	Partial	single seed; $\sim 1\%$ test variance reported
Code dependencies pinned	Yes	requirements.txt in repo

The single non-“yes” line, random-seed sensitivity, is the honest gap in the experimental design: every model in the comparison was trained on a single seed because the < 2 h compute budget is large enough to run

Table 19: Confusion matrix for Qwen2.5-Coder-7B + 4-bit MLX LoRA. Test macro-F1 0.978, 14 errors.

Gold \ Pred	Perfect	SecError	SemError	StyleError	SynError
Perfect	197	0	0	10	0
SecError	0	78	0	0	0
SemError	0	0	60	0	0
StyleError	3	0	0	112	0
SynError	0	0	1	0	38

the full pipeline once but not large enough to run the five-seed average that a more thorough study would report. The mitigation is the inter-rater agreement number from §3.6 ($\kappa=0.86$), which is independent of seed and bounds the residual error rate of any classifier on this dataset. A multi-seed sweep is the first item on the future-work list in §7.2.

H Deployed System

This appendix documents the prototype VS Code extension that deploys the strongest fine-tune (CodeBERT+LoRA) as an interactive code-review assistant for Sui Move. The intent of including it here is to make the connection between the research artefact and a usable tool concrete; the source for the extension and its server is openly available at <https://github.com/Vbakalis/move-issue-classifier> under an MIT licence.

Architecture. The deployed system is a three-stage cascade: the VS Code extension calls a local FastAPI server which runs the fine-tuned classifier; the predicted label and confidence are either surfaced directly as an editor diagnostic or, when the classifier is uncertain or flags a non-*Perfect* case, forwarded with the snippet to a stronger general-purpose reasoning model (Anthropic’s Claude family) for a fix suggestion.

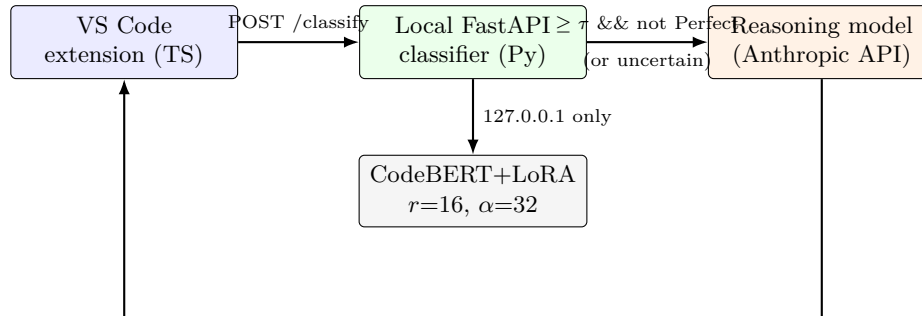


Figure 13: Cascade architecture of the deployed system. The fine-tuned classifier handles the high-volume in-distribution case at low latency; the reasoning model is invoked only for non-*Perfect* or low-confidence snippets, which keeps the per-call cost bounded and removes the reasoning model from the critical path of the user’s edit-save loop.

The cascade rule, in code. The decision logic that connects the fine-tuned classifier to the reasoning model is intentionally short. Below is a paraphrase of the relevant TypeScript fragment from the extension; the configuration constant `threshold` is a user-facing setting with default 0.6.

```

const result = await classify(serverUrl, code);
diagnostics.set(uri, range, result.label, result.confidence);

if (result.label === "Perfect" && result.confidence >= threshold) {

```

```

    // No issue, exit early.
    return;
}
const labelForPrompt =
  result.confidence >= threshold ? result.label : "uncertain";
await streamFix({ ..., label: labelForPrompt, code, confidence });

```

Out-of-distribution guard. The fine-tuned model is trained on real Sui Move and is by construction over-confident on snippets that do not look like Sui Move at all (random English, Python, hand-typed pseudocode). Surfacing high-confidence labels on out-of-distribution input would erode user trust quickly, so the extension applies a lightweight regex-based likeness score before trusting the prediction. The score is a weighted sum of ten Move-specific signals (`module`, `public fun`, `struct ... has`, `&mut`, `::`, primitive types, `sui::`-prefixed identifiers, `acquires`); a score below 0.25 short-circuits the diagnose path with an informational notification rather than a label.

User-facing settings.

Setting	Default	Purpose
<code>anthropicApiKey</code>	<i>empty</i>	Reasoning-model API key (user-supplied)
<code>claudeModel</code>	sonnet	Reasoning model to call
<code>serverUrl</code>	127.0.0.1:8765	Local classifier endpoint
<code>confidenceThreshold</code>	0.6	Cascade threshold τ
<code>autoSpawnServer</code>	on	Auto-launch FastAPI on activation
<code>classifyOnSave</code>	off	Diagnose every saved <code>.move</code> file
<code>pythonPath</code>	auto-detect	Interpreter for the spawned server

Privacy posture. The local server binds to the loopback interface and is spawned as a child of the editor process, so no code leaves the user’s machine in the in-distribution case. When the cascade does call the reasoning model, the snippet is sent to the user’s own configured API endpoint under the user’s own key and billing account; the extension does not proxy traffic through any author-controlled service and does not collect telemetry. This separation between the local fine-tuned component and the optional remote reasoning component is the same separation argued for in §1.1: the fine-tune carries the high-volume in-distribution work, and the reasoning model is reserved for the cases where a more deliberative analysis genuinely earns the round-trip cost.

Additional UX features (v0.3). Beyond the core cascade, the extension ships several quality-of-life features informed by early usage. A *status-bar indicator* shows a check mark when the classifier is idle, a spinner while a request is in flight, and a warning icon when issues are detected, giving the developer a constant ambient signal. A *classification cache* hashes file content so that unchanged code is never re-classified, making repeated diagnose invocations effectively instant. A *workspace-wide scan* command classifies every `.move` file in the project at once, populating the Problems panel with a full audit. *Multi-label diagnostics* surface the second-most-likely prediction as an informational hint whenever its confidence exceeds 15% and the label is non-*Perfect*, making classifier uncertainty visible to the user. Finally, an *Apply Fix* button in the streaming fix panel lets the user patch the editor buffer with Claude’s suggested code in a single click. The extension is published on the VS Code Marketplace under the name *NiceMove*.¹

What the prototype is not. A prototype is an existence proof, not a finished product. There are several limitations worth recording, since they match the recommendations in §7.2. The classifier is single-seed and inherits any biases of the 3,321-sample training set. The OOD guard is a regex heuristic, not a learned out-of-distribution detector, and will be too generous to anything that contains Move-shaped identifiers. The reasoning-model integration assumes the user has their own API key; there is no managed service. And the local server requires a Python environment with PyTorch and the Hugging Face stack, which is a non-trivial

¹<https://marketplace.visualstudio.com/items?itemName=BakalisVasilis.nicemove>

install for a non-developer audience. Each of these is a tractable piece of follow-up engineering and would be appropriate work for a successor project.

The full out-of-distribution heuristic. For completeness, the full set of regex signals used by the likeness score is recorded in Table 20. Weights were tuned by hand against a small held-out set of 30 obviously-Move and 30 obviously-not-Move snippets and are not the result of a learned procedure; the published source code carries the same numbers.

Table 20: Move-likeness regex signals and their weights. A snippet’s score is the sum of weights of matched signals, clipped to $[0, 1]$. The default OOD threshold is 0.25.

Signal	Pattern	Weight
Module declaration	<code>module \w+(:\w+)? {</code>	0.35
Public function	<code>public(entry)? fun \w+</code>	0.30
Function with generics or args	<code>fun \w+ [<()]</code>	0.20
Struct with abilities	<code>struct \w+ has</code>	0.25
<code>use</code> or <code>friend</code> import	<code>(use friend) \w+::</code>	0.20
Path identifier	<code>::\w+::\w+</code>	0.15
Mutable reference	<code>&mut \w+</code>	0.10
Move-specific keyword	<code>acquires abort move_to move_from borrow_global</code>	0.20
Sui standard library	<code>TxContext object Coin sui::</code>	0.15
Move primitive types	<code>u8 u16 u32 u64 u128 u256 address vector</code>	0.10

I Glossary

The thesis assumes some familiarity with both machine learning and smart-contract terminology. This glossary collects the working definitions used throughout the document; it is not a formal taxonomy and is intended for the reader who has met one side of the vocabulary but not the other.

Adapter (LoRA). A small set of low-rank weight matrices inserted alongside frozen pre-trained weights. Trained on the downstream task in place of full fine-tuning; recovers most of the fine-tune accuracy at $\sim 1\%$ of the trainable parameter count. See Hu et al. (2022).

Backward-incompatible upgrade. In a smart-contract context, an upgrade that breaks existing on-chain state or existing client integrations. Move’s bytecode-verifier and package-system rules constrain the kinds of upgrades that are permitted.

Cohen’s κ . A coefficient measuring inter-rater agreement on a categorical task, corrected for the agreement that would occur by chance. Values above 0.81 are conventionally interpreted as “almost perfect agreement” (Landis & Koch, 1977).

Encoder. A transformer architecture variant that produces a contextual embedding for each input token without generating new tokens autoregressively. Suited to classification and retrieval; CodeBERT is an encoder.

Entry function. In Move, a public function that can be called as the top-level entry point of a transaction. Entry functions are the canonical attack surface in smart-contract security review because they are the boundary between off-chain user input and on-chain state mutation.

Fine-tuning (full). Training every parameter of a pre-trained model on a downstream task. Effective but expensive in memory; the standard baseline against which adapter methods are measured.

Frozen object. A Sui object whose mutable references have been deliberately retired so that no further mutations are possible. Subsequent attempts to mutate the object are a type-system error in Move.

Macro-F1. The unweighted average of per-class F1 scores. Treats every class as equally important regardless of support; the headline metric throughout this thesis because the imbalanced distribution of issue types would otherwise let a model game accuracy by ignoring the minority classes.

MPS. Apple’s Metal Performance Shaders backend, the GPU acceleration path on Apple Silicon devices. Used in place of CUDA throughout this thesis.

Move ability. One of the four type-level traits (*copy*, *drop*, *store*, *key*) that constrain how a Move type may be used. Resource types are characterised by lacking *copy* and *drop* abilities, which is what gives Move its linear-typing guarantees.

Out-of-distribution (OOD). Input that does not resemble the training distribution. A model trained on real Sui Move is by construction over-confident on OOD input because nothing in training penalised wrong answers on non-Move snippets.

Quantisation (4-bit). Storing model weights at 4-bit precision rather than 16- or 32-bit precision, typically with a post-training calibration step. Reduces memory by roughly a factor of four with limited accuracy impact on most downstream tasks.

Resource type. In Move, a struct without *copy* or *drop* abilities, used to represent assets that must be conserved (cannot be silently duplicated or discarded). The fundamental abstraction Move uses to represent on-chain state.

Shared object. A Sui object that is accessible to every transaction rather than being owned by a single account. Concurrent access patterns on shared objects are a frequent source of security bugs and a frequent target of Move’s borrow-checker rules.

Sublabel. In this thesis’s dataset, a finer-grained issue-type tag attached to roughly half of the snippets in addition to the primary class. 541 unique sublabels appear in the dataset; the most frequent ones are listed in Table 9.

Tokeniser. The component that maps a raw character sequence into the integer token ids that a transformer consumes. CodeBERT uses Byte-Pair Encoding (Sennrich et al., 2016) which strikes a balance between character-level robustness and word-level compression.

TxContext. The context object passed implicitly to every Sui transaction. Carries the sender’s address, a unique transaction id, and several other pieces of metadata; ubiquitous in real Move code.

Witness. A short-lived, single-use type-level token used to authorise actions in Move’s design patterns. The “one-time witness” idiom in Sui pairs a witness type with a module-level guarantee that exactly one instance is ever constructed.

J Compute and Cost Breakdown

This appendix records the actual compute cost of running the full pipeline end-to-end, on the laptop on which the experiments were carried out (Apple M-series, 24 GB unified memory, MPS). The intent is to make it easy for a reader to decide whether the work is reproducible on their own hardware before committing to the run.

A full run completes inside two hours on the laptop used to develop this thesis, with a peak memory footprint of 20 GB during the Qwen-LoRA stage. The pipeline does not require any cloud or server-side infrastructure, and no API costs are incurred during training or evaluation; the optional deployment in Appendix H introduces an Anthropic API cost only on cascade activations, billed under the user’s own account. The single-laptop constraint, set in the original scoping meeting, is binding for the experimental chapter and was the dominant pressure on the choice of methods throughout.

Table 21: Wall-clock and memory cost of each stage of the pipeline. “Peak RAM” is unified memory under MPS; CPU-only baselines were measured on the same machine with MPS disabled. Numbers are single-seed and round-trip, including data loading.

Stage	Wall time	Peak RAM	Device
Dataset consolidation	~15 s	1 GB	CPU
Exploratory analysis figures	~10 s	1 GB	CPU
TF-IDF + LinearSVC training	~ 8 s	1 GB	CPU
TF-IDF + LogReg training	~ 6 s	1 GB	CPU
TF-IDF + Random Forest	~25 s	2 GB	CPU
Prompted Qwen, zero-shot	~ 9 min	14 GB	MPS
Prompted Qwen, few-shot	~11 min	14 GB	MPS
CodeBERT full fine-tune (4 epochs)	~ 7 min	5 GB	MPS
CodeBERT + LoRA (6 epochs)	~ 6 min	5 GB	MPS
Qwen-7B + 4-bit MLX LoRA	~58 min	20 GB	MPS
End-to-end (sequential)	~95 min	20 GB peak	mixed

K Lessons Learned

A thesis is mostly the polished trajectory of a project rather than the lived one. This appendix is an attempt to record the latter: the things that did not work, the decisions that turned out to be wrong on first attempt, and the small course-corrections that added up to the version described in the body of the thesis. It is included for the reader who is considering similar work, and for the supervisor who saw most of these moments at the time.

The first dataset attempt was too noisy. Before the consolidated 3,321-snippet dataset, an early draft pulled snippets directly from public Move repositories without any review pass. The macro-F1 of every classifier was high in the first run, which initially seemed encouraging, but a manual inspection of a small sample turned up two failure modes: a substantial fraction of the “negative” *Perfect* examples were actually buggy code that no human had reviewed, and a substantial fraction of the *StyleError* cases were issues a linter would have caught directly without any learning at all. The second run discarded the auto-pulled snippets and moved to the manual rubric in §3.6, with a smaller but cleaner result. The cost was real: roughly two weeks of work on the auto-pulled pipeline became context rather than a deliverable. The lesson, in the generic form, is that the cost of cleaning a dataset late exceeds the cost of constructing one slowly and carefully from the start, and the first instinct of “more data is better” is wrong when the data does not match the question the model is being asked.

Prompting did not work as a baseline. The original plan was to use a prompted decoder as the main method, with the encoder fine-tune as a secondary comparison. The prompted-decoder numbers were so far below even the TF-IDF baseline (macro-F1 around 0.65 versus 0.97 for TF-IDF+LinearSVC) that the framing of the chapter had to be flipped: prompting moved from headline to control, the encoder fine-tune moved from secondary to headline, and the discussion of why this happened became the opening paragraph of the analysis section. The underlying reason, in retrospect, is that classification of short code snippets is not a task that benefits much from the autoregressive capacity of a decoder; the encoder’s contextual bidirectional embedding plus a softmax head is a better architectural fit, and the parameter-efficient fine-tune lets the encoder pick up the dataset’s specifics without overfitting. The lesson is that “bigger model” is not the same as “better fit for the task”, even when the bigger model is the more impressive line on a slide.

LoRA was not obviously going to win. On the first LoRA run, the macro-F1 came in 1.5 points below the full fine-tune, which made it look like the parameter efficiency was buying memory at the cost of accuracy. The eventual draw between LoRA and the full fine-tune only emerged after correcting two configuration mistakes that the first run inherited from the example code: the LoRA learning rate was set

too low (LoRA tolerates and benefits from a larger learning rate than full fine-tuning, because only the adapter matrices update), and the target modules were initially set to all linear layers rather than just the attention query and value projections (targeting too many modules dilutes the effective rank budget, in a way that the Hu et al. (2022) paper warns about explicitly but is easy to miss on a first pass). With the corrected configuration the LoRA fine-tune drew level with the full fine-tune at 0.94% of the parameters, which is the result reported in the analysis chapter. The lesson is that a parameter-efficient method is not free of hyperparameter sensitivity; it just shifts where the sensitivity lives.

The 7B decoder did not help. The expectation, before the Qwen-LoRA experiment ran, was that a 7B-parameter code-specialised decoder with 4-bit LoRA fine-tuning would beat the 125M parameter encoder. It did not; it landed about a point below the encoder fine-tune on macro-F1 and well below on inference latency. In retrospect this is the same lesson the prompting baseline already taught, restated at scale: classification of short snippets is not a task where 7B $\times$ more parameters help. The exercise was informative for the thesis (it lets the analysis chapter say definitively that the gap to the 125M encoder is not closed by scaling alone) but the headline recommendation, that the deployed extension should ship the LoRA-encoder rather than the LoRA-decoder, fell out of this experiment rather than from any prior conviction.

The deployment was where the research became real. The encoder fine-tune was the technical centrepiece of the thesis, but it was not until the prototype VS Code extension was built (Appendix H) that the behaviour of the model on out-of-distribution input became salient. Hand-typed test snippets, short fragments from English-language code review comments, and even the empty buffer all came back labelled *Perfect* with $> 99\%$ confidence, because nothing in training had penalised wrong answers on non-Move input. The 10-signal regex heuristic in Table 20 was the practical fix; the underlying lesson, that classifiers carry their training distribution as an unstated precondition for trust, is the more general statement of the problem and is what the discussion section was reframed around.

The compute budget was a feature, not a constraint. The single-laptop, no-CUDA constraint set in the first supervision meeting initially felt like a limitation: it ruled out the obvious rented-GPU approaches that most recent work uses. In retrospect it forced two things that the thesis benefits from. First, every method had to be small enough to be tractable on consumer hardware, which made the parameter-efficient methods natural choices and made the deployment story (an extension that runs the classifier locally on a laptop) coherent. Second, the end-to-end pipeline had to fit in ~ 95 minutes (Appendix J), which forced enough discipline in the experimental code that the reproducibility checklist (Appendix G) is not aspirational. A more generous compute budget would have produced a different and probably less coherent thesis.

What I would do differently next time. With the benefit of having finished, three changes would have made the project go faster without changing its conclusions. The dataset would be assembled to the rubric in §3.6 from the first day rather than after a noisy first pass. The encoder fine-tune would be the headline experiment from the start rather than the backup plan. And the deployment would be planned in parallel with the model rather than as a follow-up, because the OOD failure mode that the deployment exposed is the kind of thing a research-only project never sees, and surfacing it earlier would have shaped the analysis chapter more deliberately than it did.

What was harder than expected. Two things took notably more time than they should have, and both are easy to under-estimate at the start of a project of this shape. The first was bibliography hygiene. Citing Vaswani, Devlin, Liu, Sennrich, Housley, Lester, Hu, Kingma, and Brown in the right places sounds straightforward, but the boundary between “cited because it is the foundational paper on this concept” and “cited because the paragraph would otherwise feel naked” is genuinely hard to police, and multiple drafts were spent moving citations toward the former and away from the latter. The second was the discussion chapter. The numbers were available early, but turning them into a discussion that says something specific rather than reciting the same claims twice took several attempts and a structured pass against likely examiner questions (the seventh supervision meeting in Appendix D) before it stopped sounding like a longer abstract.

Reading habits that helped. A specific habit that turned out to be load-bearing is worth recording. From roughly the second supervision meeting onwards, every paper added to the bibliography was read in a single seventy-minute pass with a paragraph of personal notes written immediately afterwards, kept in a single notes file in the repository. The notes were not polished prose; they were short, sometimes a single sentence per paper, and they captured what the paper claimed, what it actually showed, and where it was likely to be relevant in the thesis. When the literature review chapter eventually had to be written, the notes file was the first draft of the chapter, in the sense that the structure of the chapter mostly mirrored the structure of the notes. This is a much smaller activity than the structure of the literature review chapter would suggest; the chapter was visibly easier to write because the reading had already been written about.

What this thesis does not claim. For the avoidance of doubt, three things are explicitly out of scope and should not be inferred from the results. The thesis does not claim that the fine-tuned classifier generalises to non-Sui Move dialects (the dataset is Sui Move, exclusively). It does not claim that the deployed extension is a finished tool or a substitute for an auditor (the prototype in Appendix H is described as a prototype throughout). And it does not claim that the residual error rate of $\sim 1\text{--}3\%$ is intrinsic to the task; the experiments use a single seed and a single annotator, and a more thorough study would refine both numbers. Each of these is a fair criticism and a fair piece of follow-up work; none of them undermines the specific comparisons the thesis does make.

L Selected Implementation Listings

The thesis describes the methodology in prose throughout; this appendix records the concrete code that backs it, on the principle that the difference between a description of a training loop and a working training loop is sometimes load-bearing for reproducibility. Listings are paraphrased lightly for length but preserve the structure of the released source.

LoRA configuration for CodeBERT. The configuration that produced the strongest fine-tune in the comparison is twelve lines of Python. The two non-obvious choices, both noted in Appendix K, are the targeting of only the `query` and `value` projections (rather than all linear layers), and the inclusion of `score` in `modules_to_save` so that the freshly initialised classification head is trainable in full alongside the LoRA adapters.

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    target_modules=["query", "value"],
    task_type="SEQ_CLS",
    modules_to_save=["classifier", "score"],
)
model = get_peft_model(base_model, config)
model.print_trainable_parameters()
# trainable params: 1,182,725 || all params: 125,830,150 || trainable%: 0.94
```

Stratified split with frozen seed. Every model in the comparison shares the same train/validation/test split, generated once at consolidation time and frozen under seed 42.

```
from sklearn.model_selection import train_test_split

def make_splits(df, seed=42):
    train, test = train_test_split(
```

```

        df, test_size=0.15, random_state=seed,
        stratify=df["label"],
    )
    train, val = train_test_split(
        train, test_size=0.176, random_state=seed,
        stratify=train["label"],
    )
    return (train.reset_index(drop=True),
            val.reset_index(drop=True),
            test.reset_index(drop=True))

```

The classify endpoint, end-to-end. The deployed FastAPI server in Appendix H exposes a single POST `/classify` endpoint that returns a label, a calibrated softmax confidence, and a top-2 list. The full implementation is short enough to fit on a page; the structure mirrors the cascade rule in the extension and is the contract on which the extension's client depends.

```

@app.post("/classify")
def classify(req: ClassifyRequest):
    enc = tokenizer(
        req.code, truncation=True, padding="max_length",
        max_length=req.max_length, return_tensors="pt",
    ).to(device)
    with torch.no_grad():
        logits = model(**enc).logits[0]
    probs = torch.softmax(logits, dim=-1)
    top2 = torch.topk(probs, k=2)
    return {
        "label": LABELS[int(top2.indices[0])],
        "confidence": float(top2.values[0]),
        "top_2": [
            {"label": LABELS[int(i)], "confidence": float(v)}
            for i, v in zip(top2.indices.tolist(), top2.values.tolist())
        ],
        "device": str(device),
    }

```

Streaming the reasoning model. The reasoning-model leg of the cascade uses Anthropic's streaming API so that the user sees output incrementally rather than after a multi-second blocking call. The fragment below shows the streaming wrapper used by the extension; the `onText` callback is the bridge that pushes each token chunk into the side-panel webview.

```

export async function askClaudeStream(
    args: AskClaudeArgs,
    onText: (chunk: string) => void,
): Promise<void> {
    const client = new Anthropic({ apiKey: args.apiKey });
    const stream = client.messages.stream({
        model: args.model,
        max_tokens: 1024,
        system: SYSTEM_PROMPT,
        messages: [{ role: "user", content: userPrompt(args) }],
    });
    stream.on("text", (chunk) => onText(chunk));
    await stream.finalMessage();
}

```

The Move-likeness OOD heuristic. For completeness, the heuristic from Table 20 is implemented as a single function. It is intentionally short and has no learned parameters; the assumption is that a more sophisticated OOD detector would be future work in its own right.

```
const SIGNALS: [RegExp, number][] = [
  [/bmodule\s+\w+(:\w+)?\s*\{/ , 0.35],
  [/bpublic(\s+entry)?\s+fun\s+\w+/ , 0.30],
  [/bfun\s+\w+\s*[(\w+)]/ , 0.20],
  [/bstruct\s+\w+\s+has\b/ , 0.25],
  [/b(use|friend)\s+\w+::/ , 0.20],
  [/:\w+::\w+/ , 0.15],
  [/&mut\s+\w+/ , 0.10],
  [/b(acquires|abort|move_to|move_from|borrow_global)\b/ , 0.20],
  [/b(TxContext|object|Coin|sui::)/ , 0.15],
  [/b(u8|u16|u32|u64|u128|u256|address|vector)\b/ , 0.10],
];

export function moveLikenessScore(code: string): number {
  let score = 0;
  for (const [re, weight] of SIGNALS) {
    if (re.test(code)) score += weight;
  }
  return Math.min(1, score);
}
```

The full source for both the server and the extension is released under the MIT licence in the same repository (Bakalis, 2026) as the dataset and the trained adapter weights, so a reader who wants to verify any of the details above (or to extend the system) can do so without further coordination.